Univerzitet u Beogradu
Elektrotehnički fakultet



Višebitni delta sigma modulator

Diplomski rad

Kandidat:
Mihajlo Pavlović,
br. indeksa 2021/0343

Mentor:
prof. dr. Dušan Grujić
profesor Elektrotehničkog fakulteta

Beograd, septembar 2026.

Sažetak

Kroz ovaj rad obrađena je teorija, proces projektovanja i simulacije potpuno digitalnog $\Delta\Sigma$ (*Delta Sigma*) modulatora kao i teorija koja prethodi razumevanju istog. Kako bi funkcionalnost i efikasnost bile proverene napravljene su *Python*, *C++* kao i *VHDL* simulacije.

Ključne reči: Delta Sigma, Python, VHDL, Digitalna obrada signala, DAC

Sadržaj

1 Uvod **5**

2 Teorija **6**
2.1 Signali 6
2.2 Kvantizacioni šum 8
2.3 SQNR i ENOB 10
2.4 Podrhtavanje aperture odabiranja 11
2.5 Promena učestanosti odabiranja 13
2.6 Pododabiranje 15
2.7 Konvertori 17
2.8 Delta modulatori 18

3 $\Delta\Sigma$ modulator **19**
3.1 Analiza modulatora 20
3.2 Opšti model modulatora 22

4 Filtar petlje **24**
4.1 ABCD matrice 25
4.2 Poboljšanje $SQNR$ u propusnom opsegu 26
4.3 Optimizacija polova i nula NTF prenosne funkcije 27
4.4 Bandpass NTF 28

5 Problemi u projektovanju modulatora **30**
5.1 Stabilnost modulatora 30
5.2 Tonovi u praznom hodu 31
5.3 Problemi u analognim modulatorima 32

6 Strukture filtra petlje **33**
6.1 Kaskadiranje (MASH) 33
6.2 Osnovne strukture filtra petlje 34
6.3 Error feedback 36
6.4 Projektovanje CRFBD 36
6.5 Simulacije CRFBD 39

7 Postupak projektovanja višebitnog MASH(1-1) Error Feedback modulatora **40**
7.1 Projektovanje NTF 42
7.2 Simulacije u C++ 42
7.3 Simulacije u VHDL 43
7.4 Delta-sigma na višim učestanostima 43

8 Zaključak **45**

Dodatak A Projektovanje i simulacije CRFBD modulatora **47**
A.1 Projetovanje modulatora 47
A.2 Simulacija modulatora 48

Dodatak B Projektovanje i simulacije MASH(1-1) Error Feedback strukture **51**
B.1 Projektovanje modulatora 51
B.2 Simulacije u C++ 52
B.3 Simulacije u VHDL 53

Spisak slika

2.1 Prelazak iz analognog u digitalni domen 6
2.2 Spektar odabranog signala a) bez preklapanja b) sa preklapanjem 7
2.3 Blok kvantizatora 7
2.4 Linearni model kvantizatora 7
2.5 Kvantizacioni nivoi i nivoi odlučivanja, slika preuzeta iz [2] 8
2.6 Kvantizacioni nivoi i greška kvantizacije e_{q}, slika preuzeta iz [2] 9
2.7 Funkcija gustine verovatnoće greške kvantizacije $p(e_{\mathrm{q}})$, slika preuzeta iz [2] 9
2.8 Spektralna gustina snage kvantizacionog šuma za (a) f_{s1} i (b) f_{s2} 10
2.9 Odabiranje signala T/H i S/H kolom, slika preuzeta iz [2] 11
2.10 Spektar realnog takta, slika preuzeta iz [2] 11
2.11 Promena SNR_j u zavisnosti od učestanosti, slika preuzeta sa predavanja iz predmeta Hardversko Softverska Obrada Signala 12
2.12 Promena učestanosti odabiranja koristeći se konvertorima 13
2.13 Model kompresora (a) i ekspandera (b) 13
2.14 Spektar signala kroz kompresor, slika preuzeta iz [2] 14
2.15 Spektar signala kroz ekspander, slika preuzeta iz [2] 14
2.16 Decimator i interpolator 15
2.17 Odabiranje u višim nikvistovim zonama 16
2.18 Oblasti u kojim ne dolazi do preklapanja spektralnih replika pri pododabiranju . 16
2.19 Delta modulator 18

3.1 $\Delta\Sigma$ modulator 19
3.2 $\Delta\Sigma$ ADC konvertor 20
3.3 Akumulator 20
3.4 Model modulatora u Laplasovom domenu 20
3.5 Model modulatora u z domenu 21
3.6 Amplitudska karakteristika NTF modulatora prvog reda 22
3.7 Linearni model MOD2 22
3.8 Opšti model $\Delta\Sigma$ 23

4.1 Polovi i nule L_0 i L_1 za modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda 25
4.2 Optimizovani polovi i nule L_0 i L_1 za modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda 28
4.3 Polovi i nule L_0 i L_1 za bandpass modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda 29

5.1 Opšta blok šema $\Delta\Sigma$ modulatora 30
5.2 Tonovi u praznom hodu u spektru 32

6.1 Dvostepena $MASH$ struktura, slika preuzeta iz [5] 33
6.2 Struktura sa raspodeljenom povratnom spregom a) $CIFB$ b) $CRFB$ 34

6.3 Struktura sa prenosom u napred a) $CIFF$ b) $CRFF$ 35
6.4 Error Feedback struktura 36
6.5 CRFBD struktura 37
6.6 SQNR grafici projektovanog modulatora 37
6.7 Položaji polova i nula projektovanog modulatora 38
6.8 Pojačanja između redova strukture a) pre optimizacije b) posle optimizacije . . . 38
6.9 Rezultat simulacije u C++ 39

7.1 Error Feedback struktura, ponovljena slika iz odeljka 6.3 40
7.2 $MASH$(1-1) Error Feedback struktura 40
7.3 Kompletna MASH(1-1) Error Feedback struktura 41
7.4 Optimizovana MASH(1-1) Error Feedback struktura 41
7.5 Rezultat simulacije u python-u 42
7.6 Rezultat simulacije u C++ 43
7.7 Rezultat simulacije u VHDL-u 43

Glava 1

Uvod

Digitalna obrada signala se kao posebna oblast nauke i tehnike pojavila sredinom šezdesetih godina kao rezultat razvoja digitalnih računara i tehnologije integrisanih kola. Razvoj ove dve oblasti omogućio je konstrukciju malih, brzih i jeftinih, ali moćnih digitalnih računara i pratećih specijalizovanih kola. Takvi brzi specijalizovani digitalni sistemi pogodni su za realizaciju složenih algoritama za obradu signala koji se ne mogu ostvariti pomoću klasične analogne obrade signala [1]. Signali u sistemima, pogotovo u komunikacionim sistemima podložni su degradacijama u vidu šuma, koje mogu dovesti do gubitka informacija i smanjenja kvaliteta signala. Kako bi se očuvala verodostojnost signala koji se prenosi, neophodno je da greške i šum u signalu budu svedene na minimum. Tehnika za smanjivanje šuma ima mnogo, od menjanja komponenata koje sačinjavaju kolo, preko menjanja arhitektura sistema do korišćenja metoda koje se zasnivaju na matematičkim svojstvima signala. Posebnu vrstu problema predstavlja kvantizacioni šum koji je nužna posledica prebacivanja signala iz analognog domena u digitalni. U ovom radu biće obrađena jedna od tehnika kojom se kvantizacioni šum smanjuje, a to je Delta-Sigma modulacija. Komponenta u kolu koja vrši ovaj vid modulacije je $\Delta\Sigma$ modulator.

Početak rada posvećen je teoriji neophodnoj ne samo za razumevanje $\Delta\Sigma$ modulatora, već i za razumevanje koja je njihova nužnost. Biće obrađene i generalne tehnike u oblasti digitalne obrade signala.

Treće poglavlje rada posvećeno je analiziranju modela $\Delta\Sigma$ modulatora, a četvrto poglavlje je nastavak na tu temu u vidu obrađivanja komponente oko koje se vrši projektovanje modulatora *Filtra petlje*.

Peto poglavlje teorijski obrađuje probleme na koje projektant treba da pripazi tokom i posle projektovanja.

Šesto poglavlje bavi se mogućim strukturama *Filtra petlje* i u poglavlju će takođe biti naveden osnovni primer projektovanja jedne od struktura filtra sa simulacijama takvog filtra u modulatoru.

U sedmom poglavlju je po slobodnoj volji autora izabrana konkretna arhitektura modulatora, na kojoj su zatim prikazani koraci projektovanja i optimizacije, sprovedene simulacije i analizirana teorijska ograničenja, kao i mogućnosti praktične primene.

Glava 2

Teorija

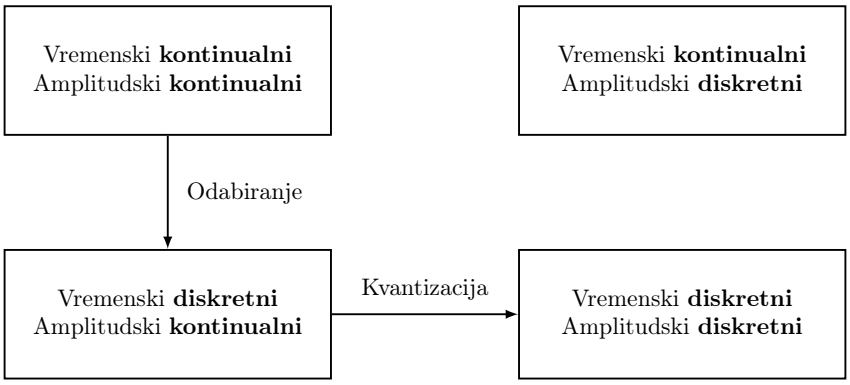
Da bismo razumeli kako $\Delta\Sigma$ modulatori funkcionišu, neophodno je razumeti šta pokušavaju da postignu. Prelaskom sa analognog domena u digitalni domen omogućavamo digitalnim kolima da brže, efikasnije i preciznije obrađuju signale. Međutim, to prebacivanje dolazi sa svojom cenom. Cena je u vidu gubitka informacija, odnosno degradacije signala u vidu šuma. Ove probleme moguće je do neke mere zaobići, ali za to je potrebno prvo se posvetiti razumevanju signala.

2.1 Signali

Pod signalom se u nauci i tehnici obično podrazumeva neka fizička veličina koja zavisi od vremena, prostornih koordinata ili neke druge nezavisne promenljive [1]. Signali u opštem smislu se mogu podeliti u 4 kategorije u zavisnosti od predstave veličine amplitude i vremenskog trenutka u kojem je definisan signal:

- Vremenski kontinualni - Amplitudski kontinualni (drugi naziv, *analogni*)
- Vremenski kontinualni - Amplitudski diskretni
- Vremenski diskretni - Amplitudski kontinualni
- Vremenski diskretni - Amplitudski diskretni (drugi naziv, *digitalni*)

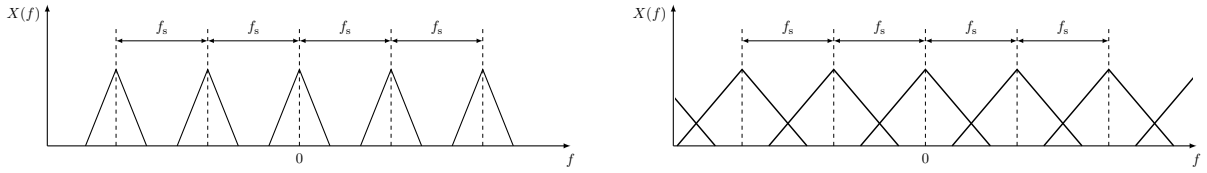
Kako bi signal diskretizovali po nekoj veličini, ili inverzno, potrebno je izvršiti neku transformaciju signala koja u nekim slučajevima može rezultovati u permanentnoj destrukciji, odnosno gubitku korisnih informacija. Proces prelaska sa vremenski kontinualnih signala u vremenski diskretne signale zove se *odabiranje*, dok se transformacija prelaska sa amplitudski kontinualnih signala na amplitudski diskretne signale zove *kvantizacija*. Oba pojma u kontekstu mogu biti nazivani i *diskretizacija* - po željenoj veličini.



Slika 2.1: Prelazak iz analognog u digitalni domen

Broj odbiraka odmeren u vremenu od jedne sekunde naziva se učestanost odabiranja f_s. Širinu opsega B *(engl. Bandwidth)* definišemo kao frekvencijski opseg u kom se nalazi naš signal. Tokom odabiranja signala, ukoliko je učestanost odabiranja neadekvatno izabrana može doći do gubitka informacija u vidu preklapanja signala u spektru.*(engl. Aliasing)* prikazano na slici 2.2. Odabiranje signala, odnosno diskretizacija po vremenu, ne dovodi do gubitka informacija ukoliko su zadovoljeni uslovi teoreme odabiranja [2], odnosno uslovi Nikvist-Šenonove *(engl. Nyquist-Shannon)* teoreme. Teorema tvrdi da svaki kontinualni signal, koji je ograničen u spektru i prethodno diskretizovan po vremenu, može biti idealno rekonstruisan iz odbiraka ukoliko za učestanost odabiranja važi (2.1).

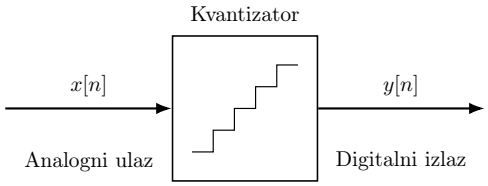
$$\mathscr{F}\left\{x_{\mathrm{a}}(t)\right\}(f) = X_{\mathrm{a}}(f) = 0 \quad \text{za} \quad |f| > B$$
$$f_{\mathrm{s}} = 1/T > 2B \tag{2.1}$$



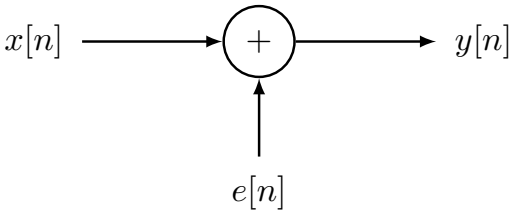
(a) (b)

Slika 2.2: Spektar odabranog signala a) bez preklapanja b) sa preklapanjem

Sa druge strane, kvantizacija je proces diskretizacija signala po amplitudi kojim se amplituda signala kontinualnog po amplitudi svodi na konačan skup vrednosti. U praksi najčešće se koristi uniformni kvantizator sa korakom Δ i izlaznom vrednošću širine od N bita. N je u ovom kontekstu prirodan broj.



Slika 2.3: Blok kvantizatora



Slika 2.4: Linearni model kvantizatora

$$y = k\,x + e \tag{2.2}$$

Diskretizacijom po amplitudi uvek dolazi do gubitaka informacija i taj gubitak se u spektru reprezentuje kao šum, preciznije *kvantizacioni šum*. Međutim, vodeći se teoremom o kvantizaciji

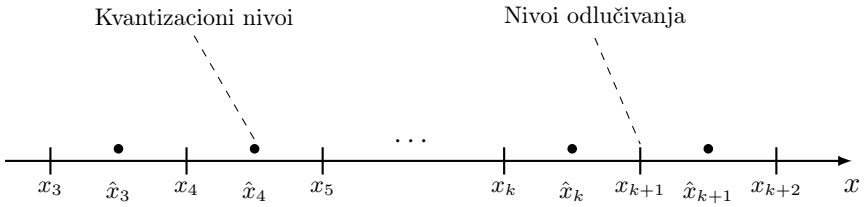
signala, kojom se nećemo baviti u ovom radu, moguće je rekonstruisati gustinu raspodele i izračunati momente originalnog signala iz kvantizovanog signala. Uslovi i rezultati teorema o kvantizaciji signala su složeni i zbog toga se veoma često koristi linearni model kvantizatora 2.4, koji kako ime kaže, ne uzima u obzir sve nelinearnosti realnog kvantizatora.

Proces prelaska iz analognog domena u digitalni obuhvata 2 koraka. Odabiranje analognog signala kako bismo dobili signal koji je diskretan po vremenu a kontinualan po amplitudi, pa onda kvantizacija signala, kojom je finalni rezultat digitalni signal.

2.2 Kvantizacioni šum

Verodostojnost signala koji se prenosi najviše određuje šum koji unose komponente u kolu. Šumova koji mogu da utiču na signal ima dosta, ali u ovom radu najviše fokusa biće na kvantizacionom šumu, koji je veći u odnosu na termički šum.

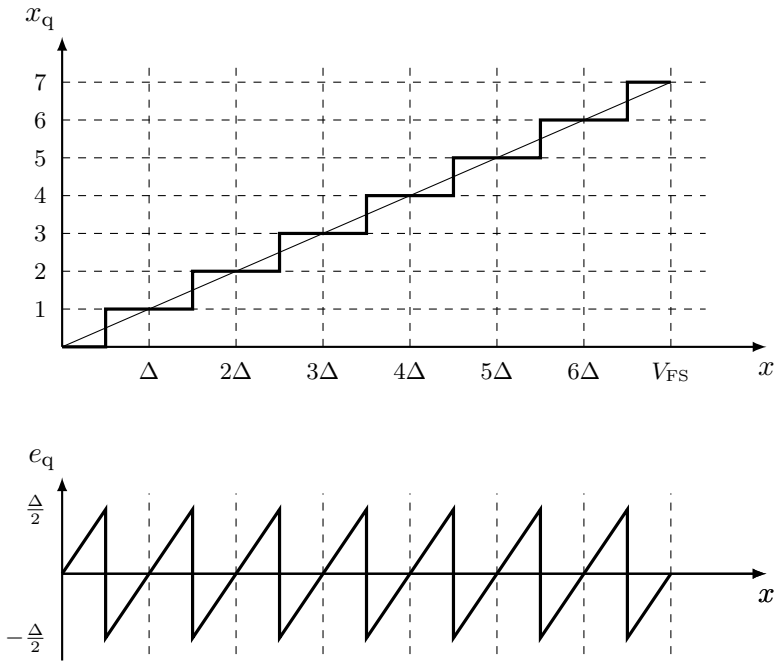
Kvantizacija signala se vrši podelom opsega ulaznog signala x u L intervala I_k, $k = 1, 2, \ldots, L$. Izlaz kvantizatora $x_q[n]$ je $\hat{x}_k$ ukoliko vrednost ulaznog signala x pripada intervalu I_k. U sistemima za digitalnu obradu signala se uglavnom koristi uniformni kvantizator gde prag odlučivanja predstavlja granicu između dva susedna intervala. Kvantizatori mogu biti simetrični i asimetrični u zavisnosti od vrednosti nivoa odlučivanja. Za N-tobitni kvantizator kažemo da ima $L = 2^N$ opsega.



Slika 2.5: Kvantizacioni nivoi i nivoi odlučivanja, slika preuzeta iz [2]

Greška kvantizacije signala $e_q[n] = x[n] - x_q[n]$ uniformnog kvantizatora, gde je Δ korak kvantizacije a $x[n]$ realna vrednost signala snimljena u istom momentu kad i kvantizovana vrednost $x_q[n]$, je u opsegu:

$$-\frac{\Delta}{2} < e_{\mathrm{q}}[n] \leq \frac{\Delta}{2}. \tag{2.3}$$



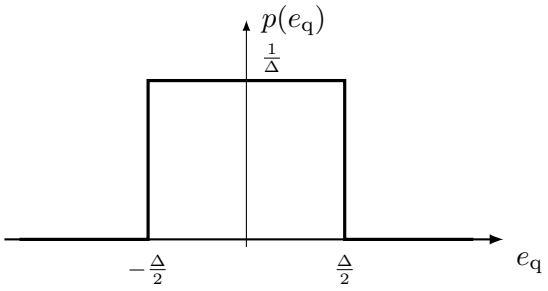
Slika 2.6: Kvantizacioni nivoi i greška kvantizacije e_q, slika preuzeta iz [2]

Grešku kvantizacionog šuma, sa razumnim pretpostavkama, možemo modelovati kao stacionarni beli šum, za koji važi raspodela $e_\mathrm{q} \sim \mathrm{Unif}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$, premda je funkcija gustine verovatnoće data sledećim izrazom:

$$p(e_\mathrm{q}) = \begin{cases} c & , -\frac{\Delta}{2} < e_\mathrm{q} < \frac{\Delta}{2} \\ 0 & , \text{inače} \end{cases} \tag{2.4}$$

U jednačini (2.4), c je konstantna. Pošto za gustinu verovatnoće važi da je njen integral na celom opsegu jednak jedinici, integraljenjem dobijamo finalni izraz:

$$p(e_\mathrm{q}) = \frac{1}{\Delta}. \tag{2.5}$$



Slika 2.7: Funkcija gustine verovatnoće greške kvantizacije $p(e_\mathrm{q})$, slika preuzeta iz [2]

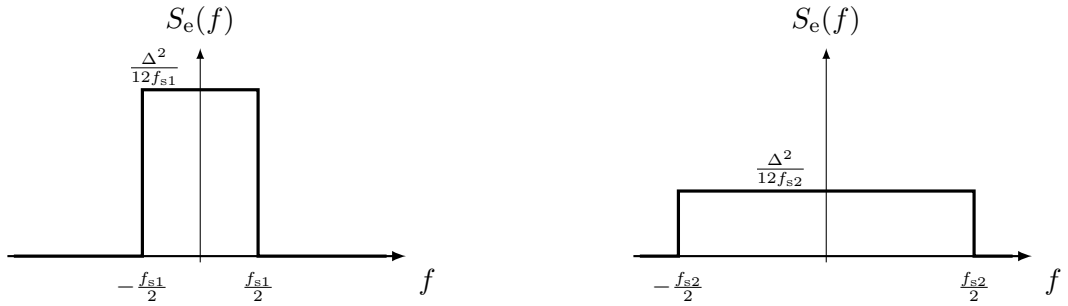
Snaga kvantizacionog šuma je jednaka varijansi belog šuma e_q i iznosi:

$$P_\mathrm{n} = \mathrm{Var}\left[e_\mathrm{q}\right] = \int_{-\Delta/2}^{\Delta/2} e_\mathrm{q}^2\, p(e_\mathrm{q})\, \mathrm{d}e_\mathrm{q} = \frac{1}{\Delta} \int_{-\Delta/2}^{\Delta/2} e_\mathrm{q}^2\, \mathrm{d}e_\mathrm{q} = \frac{\Delta^2}{12} \tag{2.6}$$

Iz jednačine (2.6) izvlačimo zaključak, na kom se zasniva ceo rad, da snaga kvantizacionog šuma ne zavisi od učestanosti odabiranja već samo od koraka kvantizacije Δ. Posledica toga je da je spektralna gustina snage kvantizacionog šuma konstantna i iznosi:

$$N_e = \frac{P_N}{f_s} = \frac{\Delta^2}{12 f_s} \tag{2.7}$$

Pri čemu je f_s učestanost odabiranja. Ovaj fenomen slikovito je predstavljen na graficima 2.8 gde su upoređeni nivoi snage kvantizacionog šuma za učestanosti odabiranja f_{s1} i f_{s2} pri čemu važi $f_{\mathrm{s2}} > f_{\mathrm{s1}}$.



(a) (b)

Slika 2.8: Spektralna gustina snage kvantizacionog šuma za (a) f_{s1} i (b) f_{s2}

2.3 SQNR i ENOB

Koliko šum utiče na koristan signal izražavamo odnosom signal-šum *(engl. Signal-to-Noise Ratio)* SNR, a koliko konkretno kvantizacioni šum utiče na koristan signal opisujemo odnosom signal-kvantizacioni šum *(engl. Signal-to-Quantization-Noise Ratio)*.

$$SQNR = \left(\frac{P_{sig}}{P_n}\right) = 12\left(\frac{P_{sig}}{\Delta^2}\right) \tag{2.8}$$

Ukoliko snagu signala predstavimo u odnosu rezolucije kvantizatora, možemo dobiti sledeći izraz:

$$SQNR = 6.02N + 1.76\,\mathrm{dB} \tag{2.9}$$

Možemo videti da sa svakim bitom rezolucije kvantizatora N više, aproksimativno dodajemo 6 dB na odnos $SQNR$. Kako povećavamo radnu učestanost konvertora tako se šum koji unosimo u kolo povećava, i realna rezolucija kvantizatora se smanjuje. Zbog toga se uvodi metrika $ENOB$ *(engl. Effective Number of Bits)* koja opisuje realnu rezoluciju kvantizatora. Prvo ćemo definisati metriku $SINAD$ *(engl. Signal-to-Noise and Distortion Ratio)* koja predstavlja još jedan odnos signal-šum ali u obzir uzima sve šumove odnosno termički šum, kT/C šum, fazni šum, nelinearnosti i distorzije [2, str. 45].

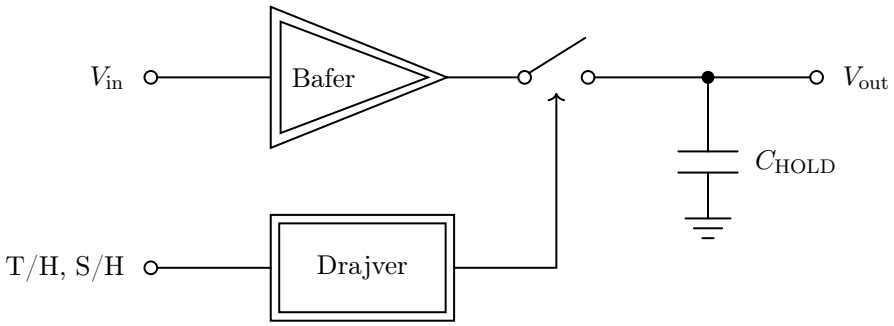
$$SINAD = \left(\frac{P_{sig}}{P_n + P_d}\right) \tag{2.10}$$

$ENOB$ u tom slučaju se može dobiti računanjem N iz jednačine (2.9), ali smenom $SQNR$ sa $SINAD$.

$$ENOB = \frac{SINAD - 1.76}{6.02} \tag{2.11}$$

2.4 Podrhtavanje aperture odabiranja

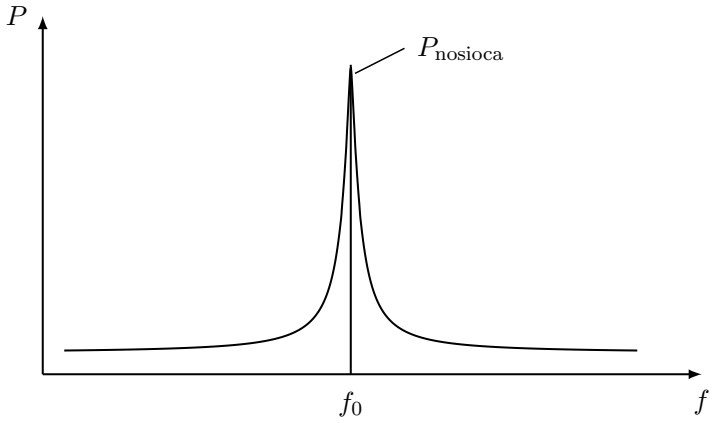
Odabiranje signala se vrši kolom za praćenje i zadršku signala *(engl. Track-and-Hold)*, ili kolom za odabiranje i zadršku signala *(engl. Sample-and-Hold)* 2.9. Ova kola vrše odabiranje pomoću prekidačkih tranzistora [2, str. 91].



Slika 2.9: Odabiranje signala T/H i S/H kolom, slika preuzeta iz [2]

U oba slučaja, kolo zadržava vrednosti signala u nekom vremenskom intervalu. Vreme isključivanja rednog prekidača, za koje se napon na kondenzatoru C_{HOLD} i dalje menja, se naziva vremenom aperture odabiranja t_a. Vreme trajanja aperture odabiranja nije konstantno, već je slučajna promenljiva određena srednjom vrednošću t_a i standardnom devijacijom $\sigma(t_a)$. Slučajno trajanje aperture odabiranja rezultuje slučajnim kašnjenjem trenutka odabiranja t_d sa standardnom devijacijom $\sigma(t_d)$. Slučajno odstupanje vremena kašnjenja trenutka odabiranja signala se naziva podrhtavanje aperture *(engl. Aperture Jitter)* i obično se obeležava sa $t_j = \sigma(t_d)$.

Podrhtavanje aperture odabiranja se u spektru realnog takta manifestuje kao šum oko učestanosti takta 2.10. Ovaj šum nazivamo fazni šum *(engl. Phase Noise)*.



Slika 2.10: Spektar realnog takta, slika preuzeta iz [2]

Odakle možemo izračunati snagu faznog šuma na opfestu Δf:

$$\mathcal{L}(\Delta f) = \frac{\int_{\Delta f}^{\Delta f+1} S(f)\,\mathrm{d}f}{P_{\text{nosioca}}} = \frac{P_{\text{šum,SSB}}(\Delta f)}{P_{\text{nosioca}}}, \tag{2.12}$$

Iz snage faznog šuma, možemo izračunati vreme podrhtavanja aperture [2, str. 96]:

$$t_{\text{j,pn}} = \frac{1}{\omega_0}\sqrt{2\int_{\Delta f_1}^{\Delta f_2} \mathcal{L}(\Delta f)\,\mathrm{d}f} \tag{2.13}$$

Ukoliko spektar signala nosioca sadrži neželjenu diskretnu komponentu (spur), doprinos vremenu podrhtavanja je

$$t_{\mathrm{j,spur}} = \frac{\sqrt{\mathcal{L}_{spur}}}{\sqrt{2}\pi f_0} \tag{2.14}$$

Ukupno vreme podrhtavanja **takta** odabiranja t_j za N nezavisnih uzroka podrhtavanja $t_{j,i}$ je

$$t_j = \sqrt{\sum_{i=1}^{N} t_{j,i}^2} \tag{2.15}$$

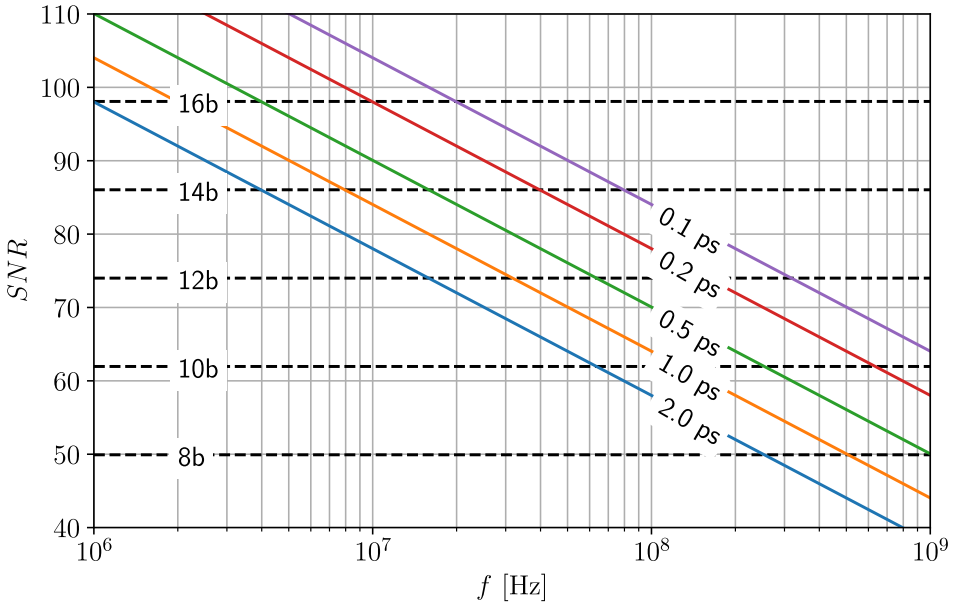
Efektivna vrednost ulaznog signala $v_{in}(t) = A\sin(2\pi f_{in}t)$ je $V_{\sin,eff} = A/\sqrt{2}$ i pod pretpostavkom da je trenutak odabiranja slučajna promenljiva može se izračunati standardna devijacija, odnosno efektivna vrednost, napona greške odabiranja:

$$V_{\mathrm{n,j,eff}} = \sigma(V_{\mathrm{n,j}}) = \sqrt{\mathrm{E}\left(V_{\mathrm{n,j}}^2\right)} = 2\pi f_{\mathrm{in}} t_{\mathrm{j}} A/\sqrt{2} \tag{2.16}$$

Odnos signal-šum usled podrhtavanja trenutka odabiranja SNR_j je:

$$\begin{aligned} \mathrm{SNR_j} &= 10\log_{10}\frac{P_{\mathrm{signal}}}{P_{\mathrm{šum}}} = 20\log_{10}\frac{V_{\mathrm{sin,eff}}}{V_{\mathrm{n,j,eff}}} \\ &= 20\log_{10}\frac{A/\sqrt{2}}{2\pi f_{\mathrm{in}} t_{\mathrm{j}}\, A/\sqrt{2}} = -20\log_{10} 2\pi f_{\mathrm{in}} t_{\mathrm{j}} \end{aligned} \tag{2.17}$$

Vidimo da se odnos signal-šum usled podrhtavanja trenutka odabiranja SNR_j opada sa porastom učestanosti ulaznog signala. Možemo nacrtati grafik SNR_j u zavisnosti od učestanosti ulaznog signala za različite vrednosti trenutka odabiranja t_j:



Slika 2.11: Promena SNR_j u zavisnosti od učestanosti, slika preuzeta sa predavanja iz predmeta Hardversko Softverska Obrada Signala

Sa grafika vidimo da isti 16-bitni ADC, sa signalom učestanošću od $100MHz$ i vremenom podrhtavanja od $t_j = 1ps$ se efektivno ponaša kao 11-bitni ADC, dok isti ADC sa vremenom podrhtavanja od $t_j = 0.1ps$ se efektivno ponaša kao 14-bitni ADC.

Inverzno možemo izračunati minimalno potrebno vreme podrhtavanja aperture odabiranja t_j formulom:

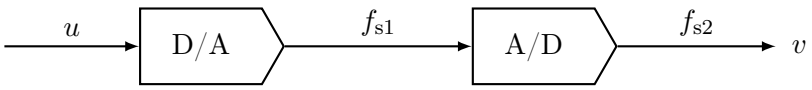
$$t_{\mathrm{j}} = \frac{10^{-\mathrm{SNR_j}/20}}{2\pi f_{\mathrm{in}}} \tag{2.18}$$

Odnos signal-šum usled podrhtavanja trenutka odabiranja SNR_j možemo sabrati po snazi sa ostalim odnosima signal-šum SNR_i koji se pojavljuju u kolu i dobiti ukupni odnos signal-šum SNR_{ukupno} (2.19). Na niskim učestanostima SNR dominantno određuje fliker šum, na srednjim kvantizacioni ili termički dok na višim podrhtavanje trenutka odabiranja.

$$SNR = (\sum_{i=1}^{N} \frac{1}{SNR_i})^{-1} \tag{2.19}$$

2.5 Promena učestanosti odabiranja

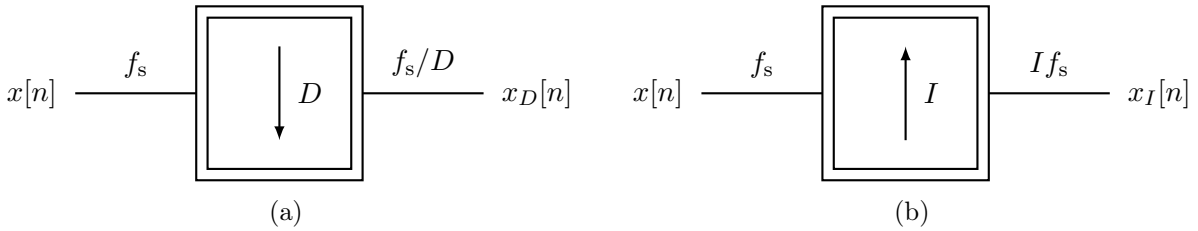
U sistemima za digitalnu obradu signala, naročito u telekomunikacijama često se ukazuje potreba da se promeni učestanost odabiranja [1]. Sistemi u kojim se koristi više učestanosti odabiranja zove se višebrzinski (engl. *multirate*) sistem. Za najjednostavniji sistem promene učestanosti možemo se koristiti *AD* i *DA* konvertorima, o kojima će više biti reči u odeljku 2.7. Digitalni signal možemo prebaciti u analogni domen pomoću *DA* konvertora, i promenu učestanosti bi vršili odabiranjem analognog signala sa *AD* konvertorom na željenoj učestanosti. Problemi u ovakvom sistemu su očigledni, izobličenja koja unosi *DAC* zbog neperfektne rekonstrukcije i kvantizacioni šum koji unosi *ADC*.



Slika 2.12: Promena učestanosti odabiranja koristeći se konvertorima

Promenu učestanosti odabiranja možemo postići potpuno digitalno, što se praktično uvek radi, pomoću *kompresora* i *ekspandera*. Detaljnije analize ovih komponenata dostupne su u [2, str. 111 - 123], u ovom radu fokusiraćemo se na to kako se promena učestanosti odražava na signal koji se obrađuje i njegov spektar. Jedino ograničenje koje imamo kod digitalnih komponenti za menjanje učestanosti je što odnos povećanja ili smanjenja učestanosti mora biti racionalan broj.

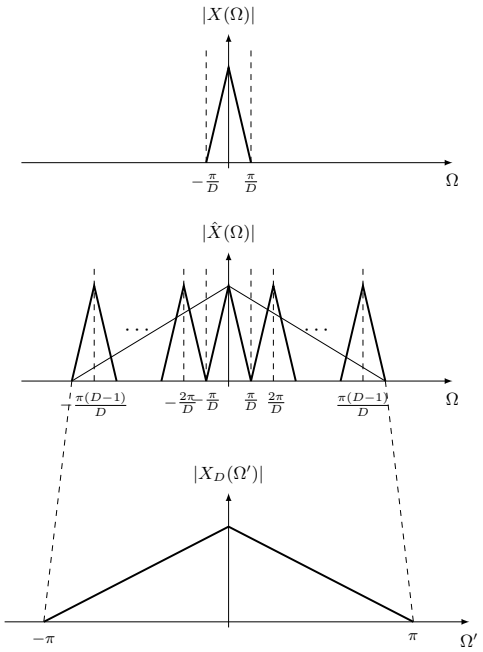
Kompresor **smanjuje** učestanost odabiranja D puta odbacivanjem $D-1$ odbiraka. *Ekspander* sa druge strane,**povećava** učestanost odabiranja I puta umetanjem $I-1$ numeričkih nula. Oba modela prikazana su na slici 2.13. Kako se ove operacije obavljaju nad digitalnim signalima, a i rad se bavi samo digitalnom realizacijom, analizu ćemo raditi nad diskretnim signalima.



Slika 2.13: Model kompresora (a) i ekspandera (b)

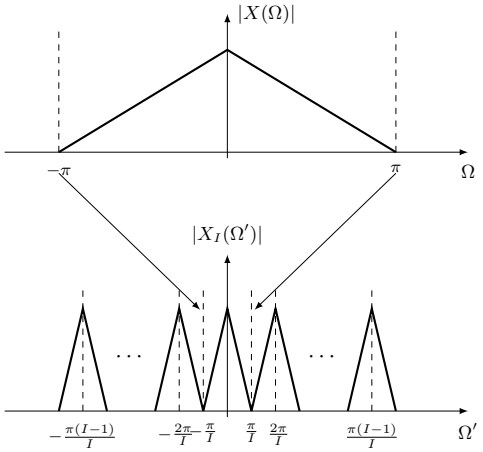
Odbacivanje odbiraka može da se posmatra u 2 koraka: Anuliranje $D-1$ odbiraka i odbacivanje nultih odbiraka. Uticaj anuliranja odbiraka se u diskretnom spektru izražava kao multipliciranje signala u kopije pomerenih za $\pm\Delta\Omega = 2\pi/D$. U ovom koraku moguće je da se spektralne

replike preklope, pa je neophodno pripaziti na to. Odbacivanjem nultih odbiraka signal oko nule „razvlačimo“ po spektru, odnosno signal koji zauzima opseg od $0 \leq |\Omega| \leq \pi/D$ sada zauzima $0 \leq |\Omega'| \leq \pi$.



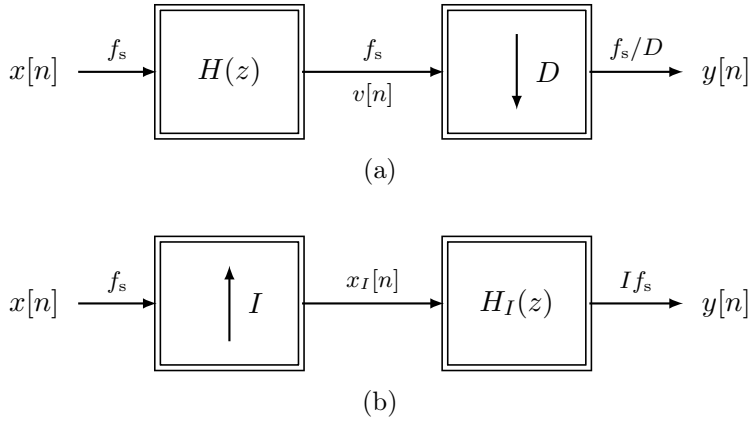
Slika 2.14: Spektar signala kroz kompresor, slika preuzeta iz [2]

Dualno odbacivanju koje razvlači spektar ulaznog signala, umetanje odbiraka ga sužava. Spektar ulaznog signala $X(\Omega)$ u opsegu digitalnih kružnih učestanosti $(-\pi,\pi)$ se slika u opseg $(-\pi/I,\pi/I)$.



Slika 2.15: Spektar signala kroz ekspander, slika preuzeta iz [2]

Sa slike 2.15 vidimo da u spektru imamo kopije spektra, koje mogu da nam smetaju pri obradi signala, čak u slučaju kompresora te kopije se mogu preklopiti i trajno nam oštetiti koristan signal. Kako bismo bili sigurni da neće doći do problema, umetanjem digitalnih niskopropusnih filtara, ispred kompresora ili iza ekspandera, dobijamo tačno jednu kopiju signala koji nam je potreban. Na ovaj način dobijamo *decimator* i *interpolator* koji se koriste kao komponente u sistemima u kojima je potrebna promena učestanosti.



Slika 2.16: Decimator i interpolator

Praktično rešenje višestepenih interpolatora i decimatora neće biti razmatrano, ali treba napomenuti da je moguća efikasna promena učestanosti sa malim brojem komponenti sa polifaznim strukturama. Takođe, moguće u modifikacije redosleda filtara i komponenti za promenu učestanosti, ali to takođe nije tema ovog rada pa je preporuka da se konsultuje literatura [2].

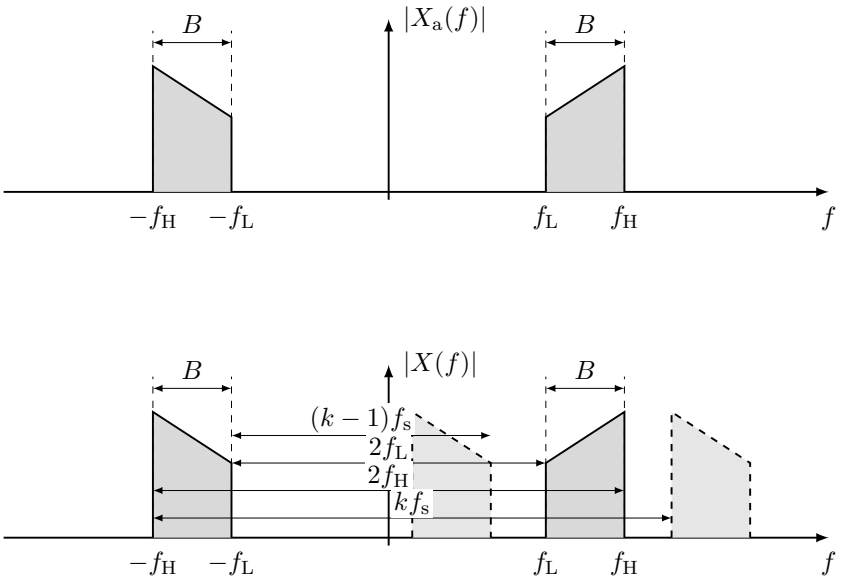
2.6 Pododabiranje

U odeljku 2.1 videli smo da kako ne bi došlo do nepovratnog gubitka informacija u signalu, potrebno je da učestanost odabiranja $f_s > 2B$. U mnogim primenama spektar korisnog signala zauzima opseg učestanosti (f_L,f_H), $B = f_H - f_L$. Samo koristeći Nikvist-Šenonovu teoremu, sa povećanjem gornje granice opsega f_H vrlo brzo bismo došli do nepraktičnih ili nemogućih vrednosti učestanosti odabiranja.

Ovaj problem možemo rešiti tehnikom odabiranja u višim Nikvistovim zonama, odnosno tehnikom pododabiranja (engl. *undersampling*). Naime, pošto spektar odabranog signala $X(f)$ sadrži skalirane i pomerene spektralne replike spektra kontinualnog signala $X_\mathrm{a}(f)$,

$$X(f) = \frac{1}{T_\mathrm{s}} \sum_{k=-\infty}^{\infty} X_\mathrm{a}(f - kf_\mathrm{s}), \tag{2.20}$$

svaka od tih replika nosi potpunu informaciju o originalnom signalu. Uslov koji mora biti ispunjen kako ne bismo narušili integritet signala je da se replike međusobno ne preklapaju.



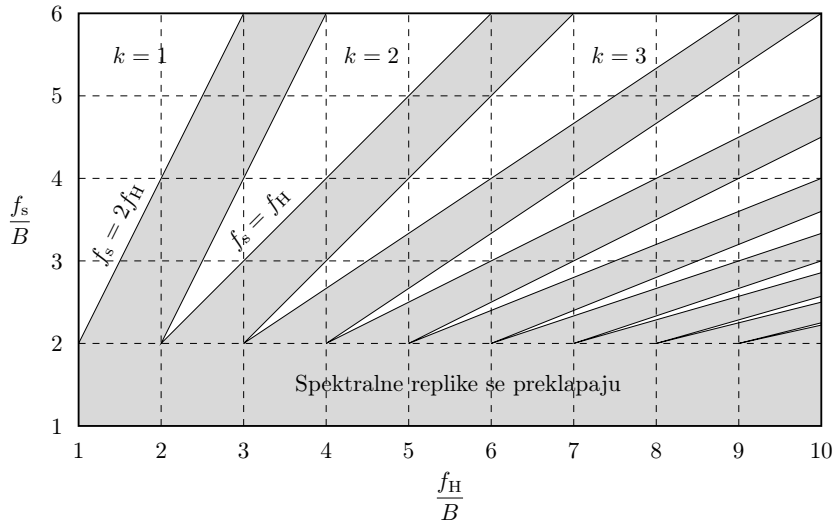
Slika 2.17: Odabiranje u višim nikvistovim zonama

Na slici 2.17 su prikazani signali koje ćemo razmatrati. Do preklapanja $(k-1)$-ve replike signala iz opsega $(-f_H, -f_L)$ sa signalom u opsegu (f_L, f_H) ne dolazi ako se ona nalazi na učestanostima nižim od f_L, dok je za k-tu repliku potrebno da se nalazi iznad f_H [2, str. 25]. Prema tome mora da važe sledeći izrazi:

$$\left.\begin{aligned}(k-1)\,f_\mathrm{s} &\le 2f_L\\ 2f_H &\le k\,f_\mathrm{s}\end{aligned}\right\} \quad\Longrightarrow\quad \frac{2f_H}{k} \le f_\mathrm{s} \le \frac{2f_L}{k-1} \tag{2.21}$$

Ukoliko u nejednakost ubacimo smenu $f_L = f_H - B$ i normalizujemo izraz na širinu opsega B dobijamo finalnu jednačinu (2.22) koja određuje oblasti u kojima ne dolazi do preklapanja spektralnih replika. Pošto su veličine normalizovane na širinu, ova nejednakost je univerzalna. Oblasti su slikovito prikazane na slici 2.18.

$$\frac{2}{k}\,\frac{f_\mathrm{H}}{B} \le \frac{f_\mathrm{s}}{B} \le \frac{2}{k-1}\left(\frac{f_\mathrm{H}}{B} - 1\right). \tag{2.22}$$



Slika 2.18: Oblasti u kojim ne dolazi do preklapanja spektralnih replika pri pododabiranju

Sa porastom broja Nikvistove zone k smanjuje se učestanost odabiranja. Maksimalnu Nikvistovu zonu određujemo preuređivanjem (2.22) i rešavanjem po k dobijamo:

$$k \leq \frac{f_H}{B} \tag{2.23}$$

Za maksimalnu nikvistovu zonu u kojoj možemo da vršimo pododabiranje uzimamo maksimalnu moguću vrednost koja odgovara uslovu (2.23) i ona iznosi maksimalan ceo deo razlomka sa desne strane nejednačine.

$$k_{max} = \left[\frac{f_H}{B}\right] \tag{2.24}$$

Prema tome, minimalna učestanost odabiranja $f_{s,min}$ u najvišoj nikvistovoj zoni iznosi:

$$f_{s,min} = \frac{2f_H}{k_{max}} \tag{2.25}$$

Pododabiranjem se izabrana replika efektivno preslikava u prvu Nikvistovu zonu, odakle se signal dalje obrađuje u digitalnom domenu. Treba voditi računa o tome da se pri preslikavanju iz parnih Nikvistovih zona spektar izvrće, odnosno dolazi do spektralne inverzije.

2.7 Konvertori

Komponente koje nam omogućavaju prelazak iz analognog domena u digitalni, ili obratno, zovu se konvertori. Analogni signal koji je obično pojava u prostoru oko nas, potrebno je digitalizovati Analognim konvertorima *ADC* kako bismo mogli signal digitalno obrađivati. Posle digitalne obrade signala, za vraćanje signala u realni prostor neophodno je digitalnu informaciju poslati Digitalnim konvertorima *DAC*.

Konvertore je moguće podeliti u 2 kategorije:

- Nikvistovi konvertori
- Nadodabirajući konvertori

Za *nikvistove* konvertore, kako ime nalaže, moraju biti ispunjeni uslovi Nikvistove teoreme odabiranja kako bi signal mogao biti verodostojno rekonstruisan. Kod *nadodabirajućih* konvertora koristi se učestanost odabiranja dosta veća od nikvistove, tipično 8-512 puta.

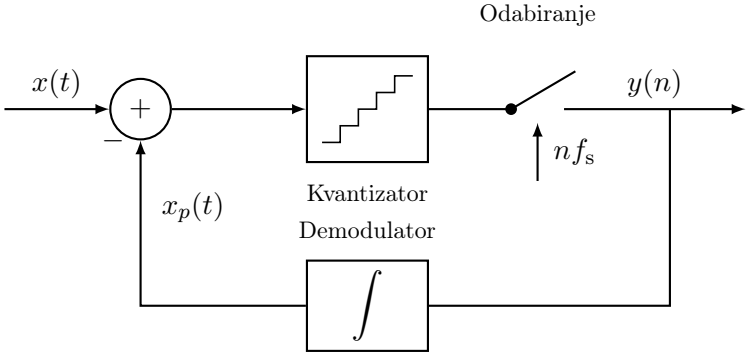
Nikvistov konvertor je uglavnom napravljen kao niz od L otpornika povezanih redno, gde je L nivo kvantizatora, kontrolisan prekidačima odnosno tranzistorima. Jako veliki problem u tim konvertorima je neujednačenost karakteristika elemenata i može se pokazati da je praktični maksimum ovakvih konvertora $ENOB = 12$ [5, str. 3]. Prednost im se nalazi u jednostavnosti pravljenja a i u činjenici da ne sadrže memorijske elemente, odnosno izlaz je direktno korelisan sa ulazom. Ukoliko bismo propustili informaciju kroz takav konvertor, u svakom trenutku mogli sa izlaza da očitamo vrednost koja odgovara ulaznoj vrednosti.

Nadodabiranje je tehnika kojom se povećava učestanost odabiranja signala kako bismo smanjili kvantizacioni šum i povećali *ENOB*. Kod nadodabirajućih konvertora uslov da je izlazna informacija korelisana sa ulaznom informacijom ne važi jer on ima memorijske komponente odnosno izlaz se generiše na osnovu svih prethodnih ulaza. Ovo je cena koju plaćamo da bismo realizovali ovakve konvertore. Nadodabirajući konvertori su u stanju da ostvare preko $ENOB = 20$. Još jedan benefit nadodabiranja je što se replike u spektru razilaze i prave veće odstojanje pa za filtriranje šuma van opsega je moguće napraviti relaksiranije filtere. Može se pokazati da dupliranjem učestanosti odabiranja postižemo poboljšanje *SQNR* od 3 dB [5, str. 29]. Naravno, povećanje odabiranja ima svoje granice i određuje ga maksimalna učestanost takta koja se može ostvariti u današnjoj tehnologiji.

2.8 Delta modulatori

Za velike rezolucije i veličinu dinamičkog opsega, *nikvistovi* konvertori opisani u odeljku 2.7 nisu odgovarajući. Povećavanjem učestanosti odabiranja približavamo se rešenju ali na posletku udaramo u plafon sa mogućnostima tehnologije. Sledeći korak unapređenja postiže se dodavanjem dodatnih komponenti u konvertor.

Za primer uzećemo *ADC*. Ukoliko digitalizujemo (*prebacimo signal iz analognog u digitalni domen*) ulazni signal $x(t)$ učestanosti odabiranja nfs razumna pretpostavka bi bila da se dva uzastopna odbirka neće puno razlikovati i naredni može biti predviđen iz prethodnog. Predviđanje se vrši dovođenjem povratne sprege u sistem. Digitalizovanje ulaznog analognog signala $x(t)$ po definiciji unosi grešku. Ako bismo takav digitalizovani signal $x_p(t)$, demodulisali i vratili kroz povratnu spregu na ulaz gde bi se oduzeo sa ulaznim signalom $x(t)$, efektivno bismo vršili digitalizaciju greške. Kako pretpostavljamo da je greška mala, potreban nam je manji red kvantizatora u kolu za digitalizaciju *DAC*. Na ovaj način funkcionišu nadodabirajući *Prediktivni Modulatori*. Najjednostavniji iz ove familije je *Linearni Delta Modulator* ili samo *Delta Modulator* čija je blok šema prikazana na slici 2.19. Naziv *Delta* dobio je jer u svojoj povratnoj sprezi računa razliku odnosno pomeraj, čiji je matematički simbol Δ.



Slika 2.19: Delta modulator

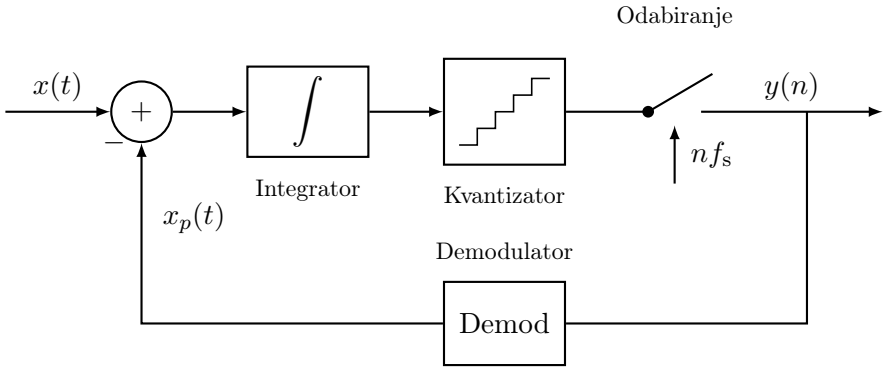
Na ovaj način, smanjili smo potreban red kvantizatora i uveli preciznije digitalizovanje signala. Modulator korišćen u *DAC* se funkcionalno i idejno ne razlikuje od modulatora pomenutog za *ADC*, razlika je što komponente modelovane u analognom domenu zamenjujemo njihovim digitalnim ekvivalentima. Kasnije u radu biće više diskusije o modelima komponenti u digitalnom domenu.

Glava 3

$\Delta\Sigma$ modulator

U prethodnom poglavlju uvedene su tehnike koje će biti korišćene u radu kao i teorija potrebna za razumevanje i opisivanje $\Delta\Sigma$ modulatora. U poglavljima se uvodi i pojam konvertora, zato što je glavna motivacija $\Delta\Sigma$ modulatora da poveća *ENOB* koji konvertor može da obradi, kako bi mogle biti opslužene primene koje zahtevaju veliki dinamički opseg i visoke učestanosti rada. $\Delta\Sigma$ modulator je nastavak na *Delta* modulator koji dodaje još jednu osobinu koja nam je veoma korisna u obradi signala.

Kvantizacioni šum je posledica parsiranja napona Vp na 2^N opsega, a kako smo videli, možemo menjati oblik spektralne gustine snage šuma kvantizatora po opsegu. Ukoliko bismo posmatrali kolo *delta* modulatora i premestili integrator iz povratne sprege u petlju signala [4], kvantizacioni šum bi se akumulirao, ali na višim učestanostima. Na ovaj način možemo pomeriti šum iz opsega gde se nalazi naš koristan signal u opseg koji nije od interesa. Ova tehnika se zove *uobličavanje šuma*. Kasnije neželjeni šum na višim učestanostima možemo filtrirati niskopropusnim filterom i u potpunosti ukloniti šum. *Sigma* u nazivu dolazi od matematičkog simbola za sumu Σ, pošto modulator sabira sve akumulirane greške.



Slika 3.1: $\Delta\Sigma$ modulator

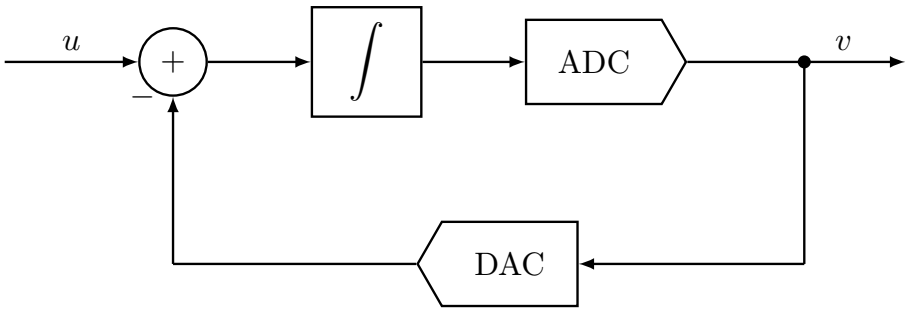
Kako $\Delta\Sigma$ modulatori koriste tehniku nadodabiranja, glavna metrika za opisivanje modulatora je nadodabirajući odnos *(engl. Over Sampling Ratio)* sa oznakom OSR. On se definiše kao odnos učestanosti odabiranja f_s i dvostruke gornje granice učestanosti korisnog signala f_B:

$$OSR = \frac{f_s}{2f_B} \tag{3.1}$$

Kada je OSR veći, to znači da je učestanost odabiranja veća u odnosu na učestanost korisnog signala. Za vrednost $OSR = 1$ modulator odabira signal na Nikvistovoj učestanosti.

Važno je napomenuti da se u praksi pojavljuju 2 vrste $\Delta\Sigma$ modulatora, analogni i digitalni. Analogni **modulator**, prikazan na slici 3.2, uvek se koristi kao **konvertor** i dolazi u paketu sa

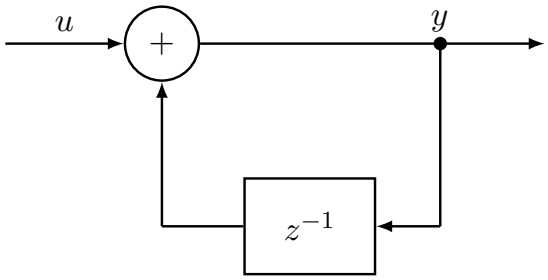
ADC i *DAC* konvertorima, dok digitalni modulator može da se koristi samostalno pošto nije neophodan prelazak nazad u analogni domen. Obe realizacije sa sobom nose svoje prednosti i mane koje će biti adekvatno razmatrane, ali u ovom radu biće projektovan i simuliran samo digitalni $\Delta\Sigma$ **modulator**.



Slika 3.2: $\Delta\Sigma$ *ADC* konvertor

Generalni model $\Delta\Sigma$ modulatora je prikazan na slici 3.1. Pošto je kvantizator nelinearan element, njegovo modelovanje je veoma komplikovano, ali za aproksimacije i projektovanje je sasvim dovoljno kvantizator modelovati kao linearni kvantizator, sa unosom statističke greške, odnosno šuma. Time rečeno kvantizator možemo modelovati kao sabirač šuma. Demodulator u povratnoj sprezi možemo modelovati kao kratak spoj.

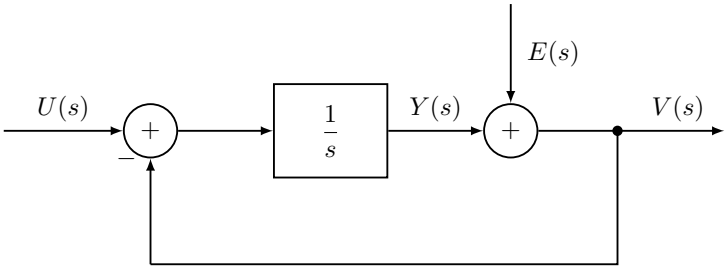
Dodatno je potrebno pojasniti modele u digitalnim sistemima. Integrator u digitalnom domenu je predstavlja običan akumulator koji je prikazan na slici 3.3, dok je linearni kvantizator samo odsecanje LSB i zadržavanje MSB bita. Za digitalna kola, u povratnu spregu dodajemo digitalno zakašnjenje jer izlaz ne sme zavisiti sam od sebe u istom taktu.



Slika 3.3: Akumulator

3.1 Analiza modulatora

Prvo ćemo analizirati sistem u Laplasovom domenu, kako bismo dobili uvid u ponašanje sistema. Integracija u Laplasovom domenu predstavlja s^{-1}. Tako da sistem možemo modelovati na sledeći način:



Slika 3.4: Model modulatora u Laplasovom domenu

Za ovakav sistem važe sledeće jednačine:

$$\begin{aligned} Y(s) &= (U(s) - V(s))\frac{1}{s} \\ V(s) &= Y(s) + E(s) \end{aligned} \tag{3.2}$$

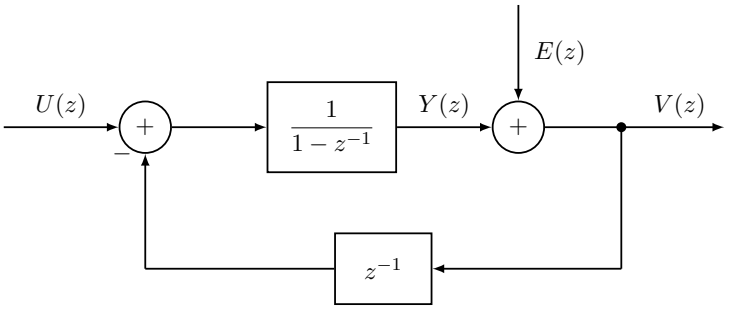
Rešavanjem sistema dobijamo izlaznu funkciju modulatora:

$$V(s) = U(s)\frac{1}{s+1} + E(s)\frac{s}{s+1} \tag{3.3}$$

Bolji uvid u sistem dobijamo vraćanjem u vremenski domen, kako je analizirano i u [3], i to je prikazano na jednačini (3.4).

$$\begin{aligned} v(t) &= (u * \mathrm{e}^{-t}u(t))(t) + e(t) - (e * (\delta(t) - \mathrm{e}^{-t}u(t)))(t) \\ &= \int_0^t u(\tau)\,\mathrm{e}^{-(t-\tau)}\,\mathrm{d}\tau + e(t) - \int_0^t e(\tau)\,\mathrm{e}^{-(t-\tau)}\,\mathrm{d}\tau \end{aligned} \tag{3.4}$$

Ulaz sistema biva propušten kroz funkciju karakteristike niskopropusnog filtra $h = \mathrm{e}^{-t}u(t)$, dok se signal greške propušta kroz funkciju $h_2 = \delta(t) - \mathrm{e}^{-t}u(t)$, koja ima karakteristiku visokopropusnog filtra. Ulaz se filtrira dok se greška **akumulira** na više učestanosti. Kako bismo projektovali sistem, potrebno je da ga posmatramo u z domenu. Sistem modelovan u z domenu prikazan je na slici 3.5 a jednačine sistema prikazane su u jednačini (3.5).



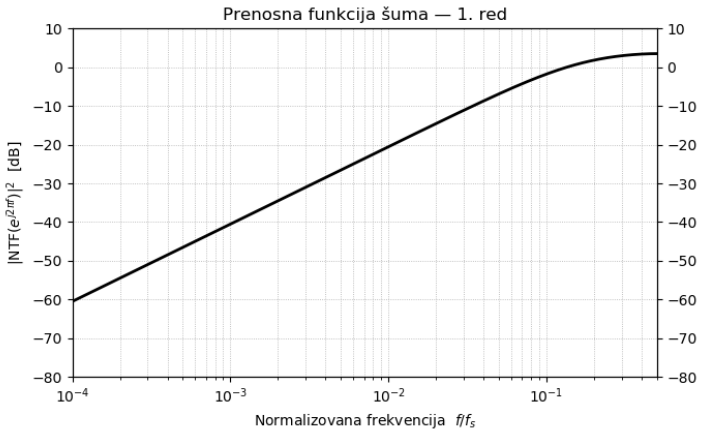
Slika 3.5: Model modulatora u z domenu

$$\begin{aligned} Y(z) &= (U(z) - V(z)z^{-1})\frac{1}{1-z^{-1}} \\ V(z) &= Y(z) + E(z) \\ &\Rightarrow \\ V(z) &= U(z) + E(z)(1 - z^{-1}) \end{aligned} \tag{3.5}$$

Funkcija prenosa pridružena uz komponentu ulaznog signala $U(z)$ naziva se Funkcija Prenosa Signala (engl. *Signal Transfer Function*) i označava se sa $STF(z)$, dok se funkcija prenosa pridružena uz komponentu greške kvantizacije $E(z)$ naziva Funkcija Prenosa Šuma (engl. *Noise Transfer Function*) sa oznakom $NTF(z)$.

Funkcije prenosa $STF(z)$ i $NTF(z)$ potpuno definišu ponašanje modulatora i modifikovanjem njih štelujemo ponašanje modulatora specifično za našu primenu. U većini slučajeva je cilj da $STF(z)$ bude što bliže 1 pošto ne želimo da unosimo distorziju u koristan signal, dok $NTF(z)$ treba da bude što bliže 0 u opsegu učestanosti korisnog signala kako bismo u tom opsegu smanjili šum što više moguće. Na slici 3.6 predstavljena je amplituda prenosne funkcije NTF prvog reda, normalizovane na opsegu, u logaritamskoj skali, koja je opisana jednačinom

(3.6) [2, str. 58]. Sa slike se vidi da je priroda NTF takva da ima malo pojačanje u opsegu i veliko van njega, naš zadatak je da tu karakteristiku poboljšamo. Optimizacija prenosne funkcije NTF je od velikog značaja za povećanje $SQNR$ u opsegu.



Slika 3.6: Amplitudska karakteristika NTF modulatora prvog reda

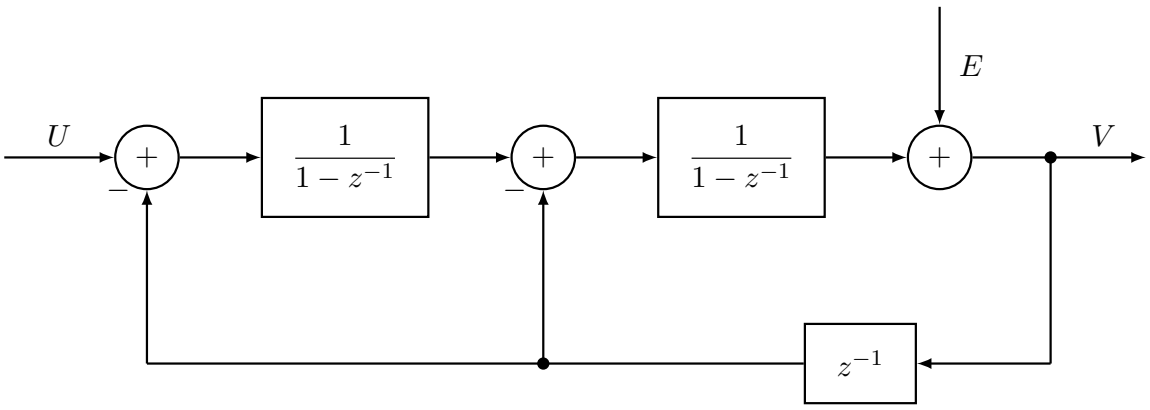
$$\left|\mathrm{NTF}\left(\mathrm{e}^{\mathrm{j}2\pi F}\right)\right|^2 = \left|1 - \mathrm{e}^{-\mathrm{j}2\pi F}\right|^2 = 2 - 2\cos(2\pi F) = 4\sin^2(\pi F) \tag{3.6}$$

3.2 Opšti model modulatora

U analizi iz prošlog odeljka, iz jednačine (3.5) funkcije prenosa definisane su kao:

$$\begin{aligned} STF(z) &= \frac{1}{z} \\ NTF(z) &= 1 - \frac{1}{z} \end{aligned} \tag{3.7}$$

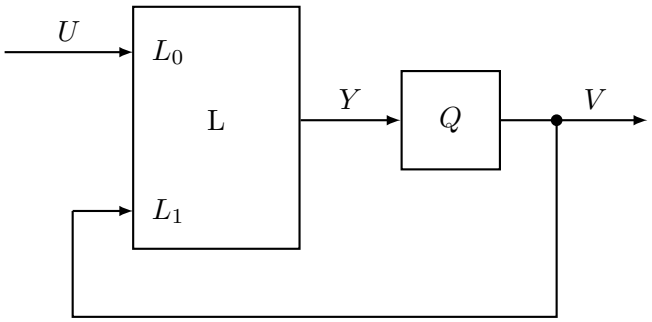
Ovakve funkcije prenosa definišu modulator prvog reda. Nalaže se da možemo dodavati više integratora i na taj način povećavati red modulatora. Povećanjem reda dobijamo više nula u funkciji prenosa šuma, čime pravimo selektivniju funkciju prenosa i time bolje oblikujemo šum. Primer modulatora drugog reda je prikazan na slici 3.7. Izlazna funkcija modulatora drugog reda je definisana u jednačini (3.8).



Slika 3.7: Linearni model MOD2

$$V(z) = U(z) + E(z)(1 - z^{-1})^2 \tag{3.8}$$

Za modulator drugog reda važi da je $STF(z) = 1$ i $NTF(z) = (1-z^{-1})^2$. Menjanjem arhitekture modulatora i povećavanjem reda modulatora možemo dobiti različite funkcije prenosa šuma i signala. U opštem slučaju, modulator može biti predstavljen kao na slici 3.8 gde su prenosne funkcije enkapsulirane u jednu funkciju koju nazivamo Filter u Povratnoj Sprezi (engl. *Loop Filter*) sa oznako $L(z)$. Karakteristike ovakvog filtera opisane su u jednačini (3.9).



Slika 3.8: Opšti model $\Delta\Sigma$

$$L_0(z) = \frac{STF(z)}{NTF(z)}, \quad L_1(z) = 1 - \frac{1}{NTF(z)}$$

$$\text{inverzno} \tag{3.9}$$

$$NTF(z) = \frac{1}{1 - L_1(z)}, \quad STF(z) = \frac{L_0(z)}{1 - L_1(z)}$$

Glava 4

Filtar petlje

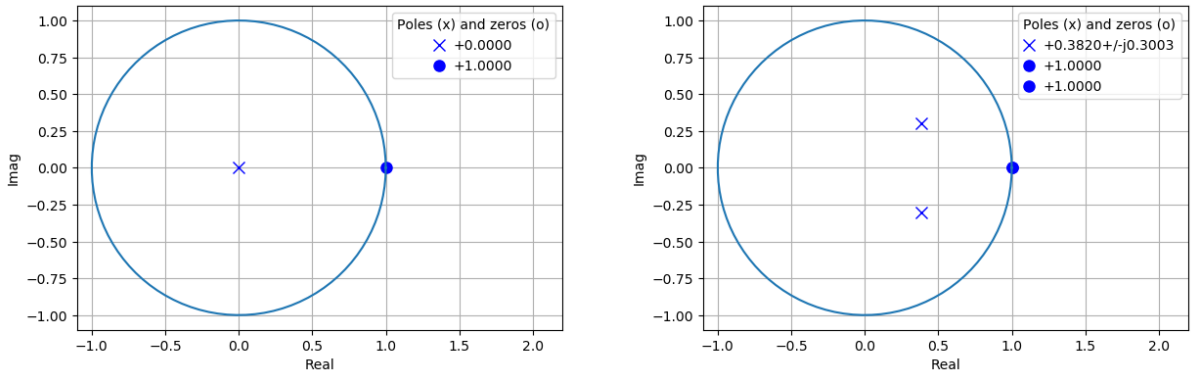
Enkapsuliranje prenosnih funkcija NTF i STF, na prvi pogled nepotrebno, nam pomaže da detaljnije analiziramo ponašanje sistema. Uzećemo za primer prenosne funkcije za kojoj smo proizvoljno izabrali $NTF = (1 - z^{-1})^N$ i $STF = z^{-k}$. Kašnjenje u vrednosti k je proizvoljno i ne utiče na analizu, ali u praksi je često potrebno implementirati dodatno kašnjenje kako bi STF bio što približniji jedinici. Prenosne funkcije Filtera u Povratnoj Sprezi date su sledećom jednačinom:

$$\begin{aligned} L_0(z) &= z^{-k}(1 - z^{-1})^{-N} \\ L_1(z) &= 1 - (1 - z^{-1})^{-N} \end{aligned} \tag{4.1}$$

Jednačina (3.9) nam sugeriše da L_1 mora biti veliko u frekvencijama signala kako bismo smanjili NTF, ali i L_0 mora biti veliko u frekvencijama signala kako bi STF bio blizu 1. L_1 i L_0 treba da imaju polove u ovom opsegu. Ukoliko analiziramo jednačinu (4.1) vidimo da se polovi L_1 i L_0 zaista nalaze na istom mestu, i to na jediničnom krugu. $N - k$ nula od L_0 se nalazi u $z = 0$ tački, dok se ostalih k nula nalazi na u beskonačnosti. Nule L_1 se nalaze u korenima jednačine $1 - (1 - z^{-1})^{-N}$ i možemo ih iskazati po jednačini (4.2).

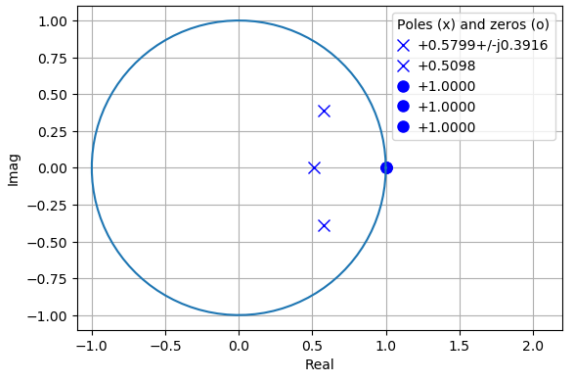
$$\begin{aligned} z_{L_1,i} &= \frac{1}{1 - e^{j\frac{2\pi i}{N}}} \\ &= \frac{1}{2}[1 + j\cot(\pi i/N)], \quad i = 0,1,...,N-1 \end{aligned} \tag{4.2}$$

Na sledećim slikama prikazani su primeri polova i nula L_0 i L_1 za modulatore reda $N \in (1,4)$.

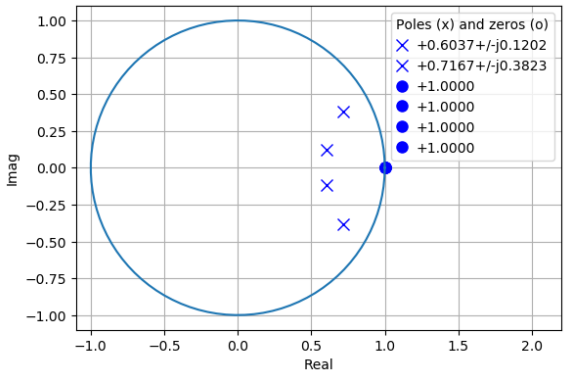


(a)

(b)



(c)



(d)

Slika 4.1: Polovi i nule L_0 i L_1 za modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda

4.1 ABCD matrice

Pošto je filtar petlje LF kauzalan, linearan i vremenski invalijantan sistem, ponašanje filtera možemo matematički opisati preko modela prostora stanja *(engl. State Space model)* za diskretne sisteme, u kontekstu zvanom $ABCD$ matrica (4.5). Model prostora stanja razdvaja diferencijalne jednačine i algebarske jednačine, često je izuzetno korisno u nizu metoda za analizu električnih kola i sistema uopčste, posebno u teoriji automatskog upravljanja [7]. Analizu ćemo započeti razlaganjem izlaza $y(n)$ filtra u povratnoj sprezi LF na diferencnu jednačinu stanja (4.3) i algebarsku jednačinu izlaza (4.4).

$$x(n+1) = A\,x(n) + B\begin{bmatrix} u(n) \\ v(n) \end{bmatrix} \tag{4.3}$$

$$y(n) = C\,x(n) + D\begin{bmatrix} u(n) \\ v(n) \end{bmatrix} \tag{4.4}$$

$$ABCD = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \tag{4.5}$$

U najprostijem slučaju, uzimajući za primer modulator prvog reda koji opisuje jednačina (3.5), izlaz $y(n)$ je dat je izrazom:

$$y[n] = u[n] - v[n-1] + y[n-1] \tag{4.6}$$

Primetimo da $v[n-1]$ i $y[n-1]$ opisuju memoriju sistema, odnosno prethodno stanje sistema. Smenom $x[n] = y[n-1] - v[n-1]$, jednačinu (4.6) razložili smo na diferencnu i algebarsku jednačinu datu:

$$\begin{aligned} x[n+1] &= y[n] - v[n] \\ y[n] &= x[n] + u[n] \\ \Rightarrow x[n+1] &= x[n] + u[n] - v[n] \\ y[n] &= x[n] + u[n] \end{aligned} \tag{4.7}$$

$ABCD$ matrica koja opisuje ovaj sistem je $A = 1$, $B = \begin{bmatrix} 1 & -1 \end{bmatrix}$, $C = 1$, $D = \begin{bmatrix} 1 & 0 \end{bmatrix}$, odnosno tablično zapisano:

$$\text{ABCD} = \left[\begin{array}{c|cc} A & B_u & B_v \\ \hline C & D_u & D_v \end{array}\right] = \left[\begin{array}{c|cc} 1 & 1 & -1 \\ \hline 1 & 1 & 0 \end{array}\right] \tag{4.8}$$

Takođe, moguće je inverzno odrediti jednačine LF preko ABCD matrice rešavanjem linearnog sistema jednačina:

$$\begin{bmatrix} L_0(z) \\ L_1(z) \end{bmatrix} = C\left(z^{-1}I - A\right)^{-1} B + D \tag{4.9}$$

Preko $ABCD$ matrica moguća je dodatna optimizacija u vidu izjednačavanja međustepenog pojačanja [8, str. 8].

Kako bismo izračunali vrednosti $ABCD$ matrice, najpre moramo izabrati strukturu jer nam ulazne i izlazne vrednosti LF zavise od toga a samim tim i matrica. $ABCD$ matrica može imati više rešenja LF čak i za iste strukture, tako da nam $ABCD$ matrice ne govore detaljne informacije LF već isključivo kako se filter ponaša kao odvojeni blok.

4.2 Poboljšanje $SQNR$ u propusnom opsegu

Analiza polova i nula nam je zgodna za analizu jer nam pokazuje kako se sistem ponaša u frekvencijskom opsegu. Doduše ne daje nam nikakvu informaciju o kvalitetnosti sistema, odnosno koliko poboljšanje $SQNR$ možemo da očekujemo.

Vratimo se na prenosnu funkciju sistema prvog reda, iz jednačine (3.5). Šum na izlazu sistema predstavljen je prenosnom funkcijom $Q(z) = E(z)(1 - z^{-1})$.Prebacivanjem u frekvencijski domen, odnosno smenom $z = e^{j2\pi fT}$, spektralna gustina šuma data nam je sledećom jednačinom:

$$N_q(f) = (2sin(\pi fT))^2 \frac{N_e}{2} \tag{4.10}$$

Gde je N_e spektralna gustina kvantizacionog šuma iz jednačine (2.7). Integraljenjem po frekvenciji u opsegu $(0, B)$, uz pretpostavku da je $OSR >> 1$ dobijamo snagu šuma u opsegu:

$$\begin{aligned} q_{rms}^2 &= \int_0^B N_q(f)df \\ &= \frac{\pi^2 P_n^2}{3OSR^3} \end{aligned} \tag{4.11}$$

Istim metodom možemo doći do snage šuma za modulator reda L za koji važi da je prenosna funkcija šuma na izlazu $Q(z) = E(z)(1 - z^{-1})^L$, i važi:

$$q_{rms}^2 = \frac{\pi^{2L} P_n^2}{(2L+1)OSR^{2L+1}} \tag{4.12}$$

Izvlačimo jako bitne zaključke iz ove analize. Poboljšanje $SQNR$ postižemo samo modifikovanjem reda modulatora i povećanjem OSR i smanjenjem koraka kvantizacije. Povećanje koraka kvantizacije se u praksi nikada ne radi, ali povećanjem broja koraka, smanjujemo šum sam po sebi, tako da to je jedan dodatni stepen slobode. Povećanjem reda modulatora dobijamo veće poboljšanje $SQNR$ u opsegu, ali i povećavamo kompleksnost sistema. Kako bi imali dobar $SQNR$, nameće se minimalna vrednost OSR koja u praksi iznosi 8.

4.3 Optimizacija polova i nula NTF prenosne funkcije

Iz prethodnih poglavlja, neko bi zaključio da je moguće samo dizajnirati prenosne funkcije sa nulama u $z = 1$ na jediničnom krugu. Ako fiksiramo OSR, red filtra N i broj koraka kvantizatora M, poboljšanja u vidu povećanja $SQNR$ postižemo modifikovanjem prenosne funkcije šuma NTF odnosno optimizovanjem njenih polova i nula. Kako će se ponašati naš sistem u propusnom opsegu u potpunosti zavisi od oblika NTF. Za primer optimizacije uzećemo NTF drugog reda $N = 2$. U prošlom poglavlju, na slici 3.6 vidimo ponašanje NTF prvog reda, ali veoma sličnu amplitudsku karakteristiku dobijamo i za NTF drugog reda, tako da analizu možemo nastaviti sa tom slikom. Amplitudsku karakteristiku prenosne funkcije, za velike vrednosti OSR u propusnom opsegu možemo aproksimirati kao:

$$\begin{aligned} |NTF| &= \frac{(1 - z^{-1})^2}{A(z)} \\ |NTF| &\approx K\omega^2 \end{aligned} \tag{4.13}$$

Gde je $A(z)$ slabljenje, a $K = 1/A(1)$ pojačanje na jediničnom krugu [5]. Pomeranjem nula od vrednosti $z = 1$ do $z = e^{\pm j\alpha}$ amplituda postaje:

$$|NTF| = K(\omega - \alpha)(\omega + \alpha) = K(\omega^2 - \alpha^2)$$

Snagu šuma u propusnom opsegu tad definišemo jednačinom (4.14). U jednačini važi da je ω_B kružna učestanost koja odgovara učestanosti propusnog opsega.

$$\begin{aligned} P(\alpha) &= \int_0^{\omega_B} |NTF|^2 d\omega \\ &= \int_0^{\omega_B} K^2(\omega^2 - \alpha^2)^2 d\omega \end{aligned} \tag{4.14}$$

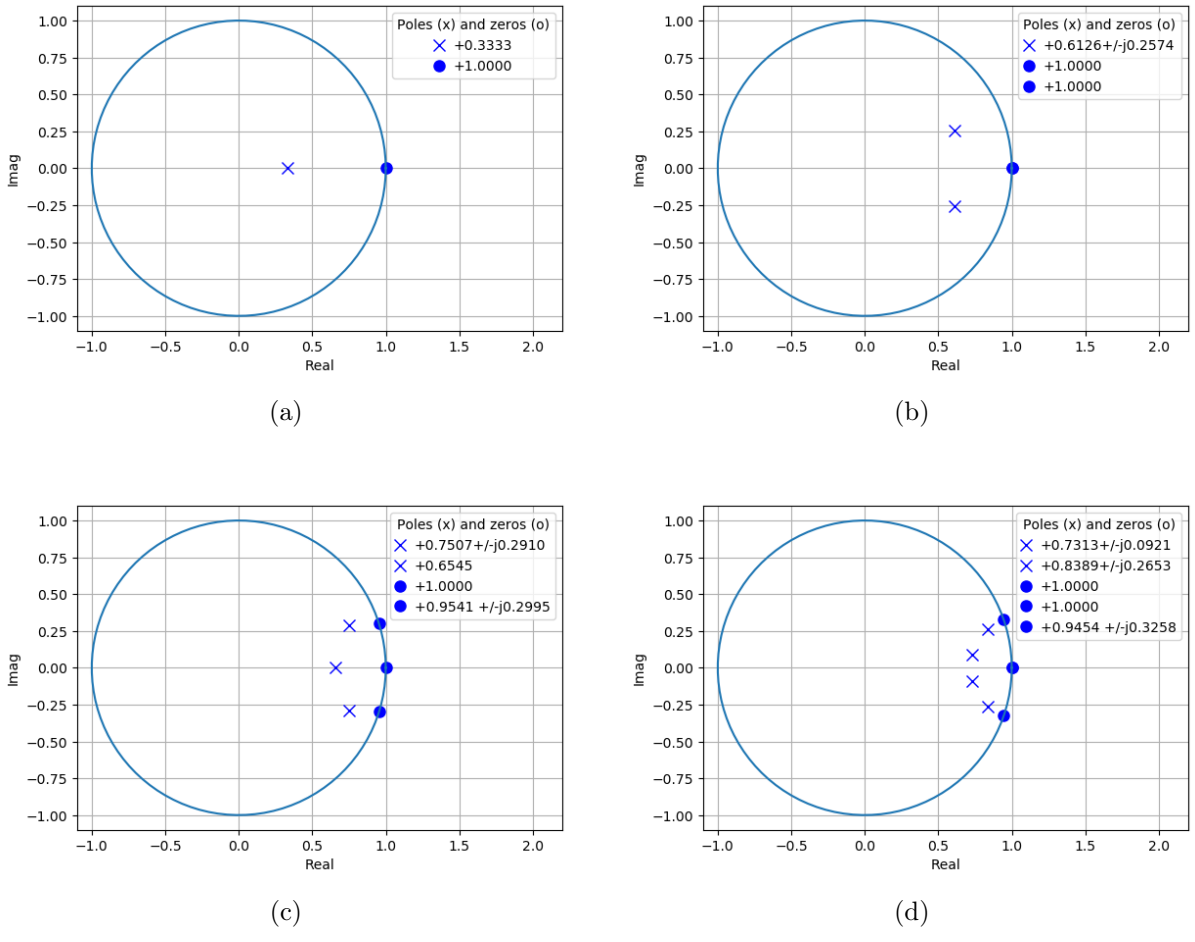
Minimiziranjem prethodne jednačine pronalazimo optimalnu poziciju nula za koju važi da je amplituda NTF u propusnom opsegu minimalna, i za drugi red NTF vrednost koja se dobija je:

$$\alpha_{opt} = \frac{1}{\sqrt{3}}$$

Očekivano poboljšanje $SQNR$ u propusnom opsegu je 3,5 dB. Za više redove NTF optimizacija se vrši na isti način, a rezultati za vrednosti $N \in (1{,}8)$ su prikazani u tabeli 4.1. Na slici 4.2 prikazana su optimizovana mesta nula i polova sa primera 4.1.

N	položaj nula, normalizovan na ω_B	poboljšanje SQNR
1	0	0 dB
2	$\pm 1/\sqrt{3}$	3,5 dB
3	$0,\ \pm\sqrt{3/5}$	8 dB
4	$\pm\sqrt{3/7 \pm \sqrt{(3/7)^2 - 3/35}}$	13 dB
5	$0,\ \pm\sqrt{5/9 \pm \sqrt{(5/9)^2 - 5/21}}$	18 dB
6	$\pm 0{,}23862,\ \pm 0{,}66121,\ \pm 0{,}93247$	23 dB
7	$0,\ \pm 0{,}40585,\ \pm 0{,}74153,\ \pm 0{,}94911$	28 dB
8	$\pm 0{,}18343,\ \pm 0{,}52553,\ \pm 0{,}79667,\ \pm 0{,}96029$	34 dB

Tabela 4.1: Položaj nula za minimalni šum u propusnom opsegu, tabela preuzeta iz [5, str. 108]



(a) (b)

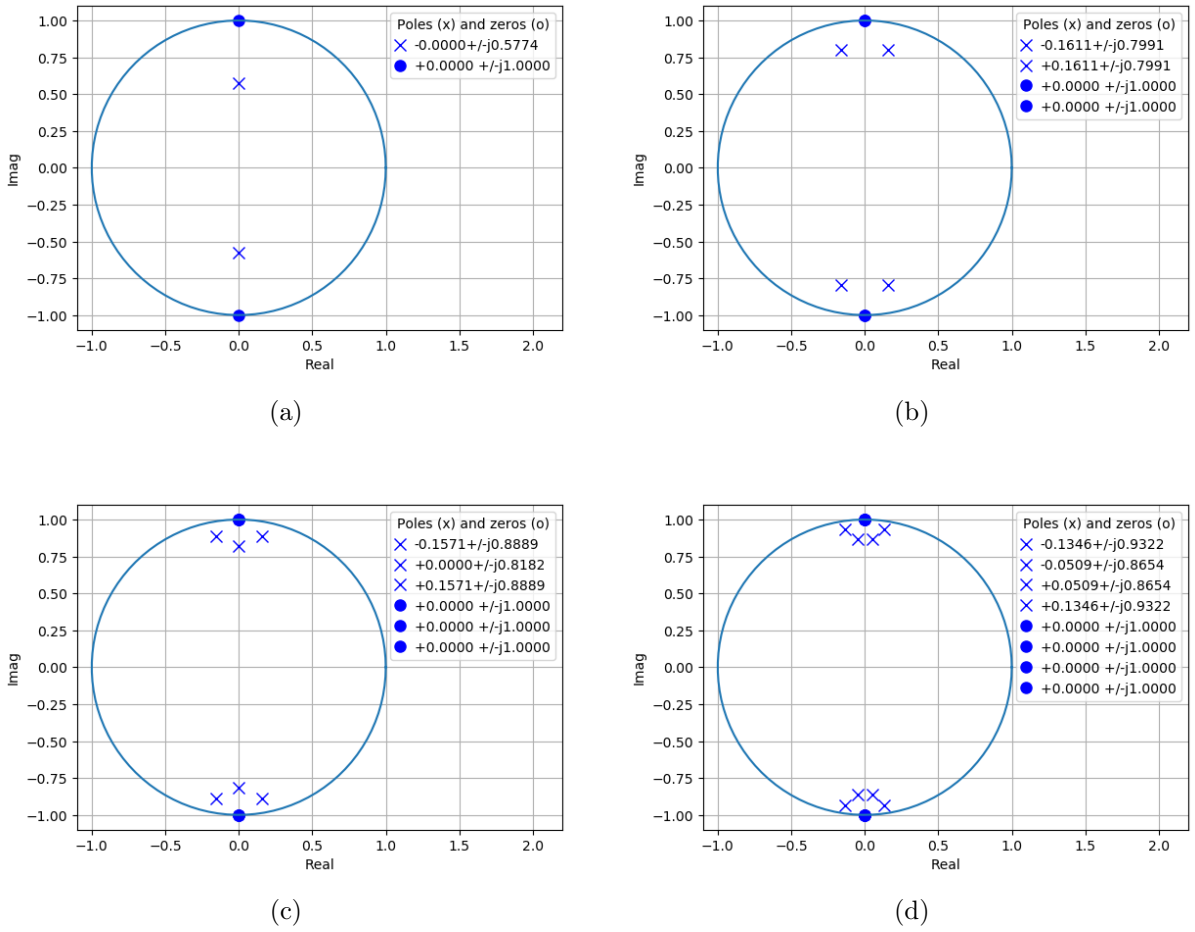
(c) (d)

Slika 4.2: Optimizovani polovi i nule L_0 i L_1 za modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda

4.4 Bandpass NTF

Za primene u kojima nije dovoljno obraditi signal samo na dc, možemo napraviti modifikaciju NTF tako da smanjuje šum u opsegu učestanosti oko f_c, u opsegu (f_L, f_H). Ovakva realizacija

modulatora zove se Bandpass $\Delta\Sigma$ modulator. Za ovakav modulator važi isto pravilo kao i za bandpass filter, modulator mora biti parnog reda. Nadovezaćemo se na primer sa slike 4.1 u kom su prikazani polovi i nule lowpass NTF, i za ovaj primer nazvaćemo ih **prototopski** NTF. Razdvajanjem nula i polova kružno za proizvoljan ugao ϕ, realizujemo bandpass karakteristiku. Za slučaj kada centriramo opseg na $f_0 = f_s/4$, smenom $z \to -z^2$, dupliramo nule prototipskog NTF čime zadovoljavamo uslov za bandpass, i N nula ovakvog NTF selimo na $z = j$ a ostalih N na $z = -j$. Bandpass filteri, projektovani na ovakav način prikazani su na slici 4.3.



(a) (b)

(c) (d)

Slika 4.3: Polovi i nule L_0 i L_1 za bandpass modulatore a) prvog, b) drugog, c) trećeg i d) četvrtog reda

Još jedna dobra modifikacija modulatora je kvadraturni modulator. Iako je on modifikacija celog sistema a ne samo filtra petlje, pomenut je u ovom odeljku jer se direktno nadovezuje na bandpass NTF. Kvadraturni modulator može biti konstruisan i sa bandpass i sa lowpass NTF ali ima frekvencijski odziv kao bandpass NTF osim što ne digitalizuje negativne vrednosti spektra, odnosno nema frekvencijski odziv simetričan oko $f = 0$. Samim tim, kvadraturni modulator je duplo efikasniji od bandpass modulatora. Još jedna dobra osobina kvadraturnih modulatora je što preklapanje u spektru, koje se kod ostalih sistema dešava za vrednosti $f > f_s/2$, kod kvadraturnih modulatora se dešava za $f > f_s$, jer su granice signala (-$f_s/2$, $f_s/2$). Na ovaj način, efektivno se duplira vrednost OSR i time dodatno poboljšava $SQNR$ u opsegu.

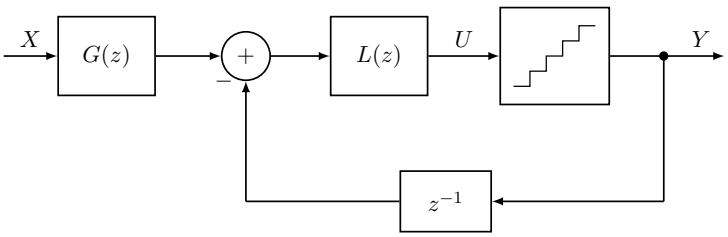
Glava 5

Problemi u projektovanju modulatora

Svi benefiti koje unosi $\Delta\Sigma$ modulator, sa sobom nose i mnoštvo problema u projektovanju. Projektovanje nije lak posao i projektant mora da se drži određenih pravila kako bismo isprojektovali dobar sistem. Problemi koji se javljaju razlikuju se u zavisnosti da li projektujemo analogni ili digitalni modulator, a ima i nekih zajedničkih problema. U ovom poglavlju razmatraćemo probleme koji se pojavljaju u obe realizacije kako bismo stekli bolji uvid i razumevanje modulatora. Jasno će biti naznačeni na koju reazlizaciju se problem odnosi.

5.1 Stabilnost modulatora

Glavni problem u projektovanju, koji odlučuje da li će modulator raditi ili ne je stabilnost. Problem stabilnosti je univerzalan i za analogne i za digitalne modulatore. Stabilnost sistema je veoma kompleksna materija i njena analiza ne dovodi do fiksnih rezultata, tako da proveru stabilnosti sistema je najbolje napraviti simulacijama. Različiti metodi određivanja stabilizacije postoje i u ovom radu obrađivaćemo Metod Promenjivog Pojačanja (engl. *Variable Gain Method*). Posmatraćemo najosnovniji model modulatora, prikazan na slici 5.1.



Slika 5.1: Opšta blok šema $\Delta\Sigma$ modulatora

Metodom Promenjivog Pojačanja, svaki nelinearni element u kolu možemo zameniti proizvoljnim pojačanjem K [6, str. 168]. Možemo izvući sada prenosnu funkciju sistema $H(z, K)$ koja ukoliko je stabilna, mora da zadovoljava uslove stabilnosti, odnosno da su svi polovi prenosne funkcije unutar jediničnog kruga za svaku vrednost K.

$$H(z) = \frac{KL(z)G(z)}{1 + z^{-1}KL(z)} \tag{5.1}$$

Ukoliko sistem nije stabilan za sve vrednosti K, ovim metodom, snimamo poslednju vrednost $K_{min} = K$ za koju je sistem stabilan. Maksimalna apsolutna vrednost na ulazu sistema za koju je sistem stabilan određujemo:

$$max = \frac{1}{K_{min}} \tag{5.2}$$

Naredne aproksimacije stabilnosti razlikuju se od reda nelinearnog elementa u kolu, odnosno od broja nivoa kvantizacije. Analiziraćemo prvo jednobitne modulatore. Može se pokazati za modulator prvog reda da je $u_{max} = 1$ [5, str. 49]. Ali tu se završava jednostavnost, što više povećavamo red modulatora, to nam je granica stabilnosti niža. Kada zađemo u analizu modulatora drugog reda, granicu ne možemo tačno odrediti [5, str. 76]. Autori iz literature preporučuju da se držimo amplitude $u_{max} < 0.9$ u slučaju modulatora drugog reda. U slučaju viših redova, stabilnost je potpuno nepredvidiva i u različitim arhitekturama modulatora dobijaju se različite vrednosti u_{max}.

Kako bismo imali neku orijentaciju, najbolje je da se držimo Liovog kriterijuma *(engl. Lee criterion)* [10] koji tvrdi:

> Proizvoljan $\Delta\Sigma$ modulator za koji važi NTF $= H(z)$
> biće najverovatnije stabilan ako važi $\max_{\omega} \left|\mathrm{H}\left(e^{j\omega}\right)\right| < 1{,}5.$ (5.3)

U kriterijumu (5.3), $\left|\mathrm{H}\left(e^{j\omega}\right)\right|$ predstavlja maksimalno pojačanje NTF na celom opsegu frekvencija, takođe poznato kao beskonačna norma $\|\mathrm{H}\|_\infty$. Lijev kriterijum nije ni potreban ni dovoljan uslov, već empirijsko pravilo: modulatori koji ga zadovoljavaju su u praksi po pravilu stabilni, ali postoje i stabilni modulatori koji ga narušavaju.

Za višebitne modulatore, važi sledeća relacija [5, str. 104]. Za modulator sa M kvantizacionih nivoa, pod uslovom da je početna vrednost $u(0)$ u opsegu linearnosti kvantizatora, stabilnost je zagarantovana za proizvoljan ulaz $u(n)$ ako važi

$$\max_n |u(n)| < M + 2 - \|h\|_1 , \tag{5.4}$$

gde je

$$\|h\|_1 = \sum_{k=0}^{\infty} |h(k)| , \tag{5.5}$$

a $h(k)$ je inverzna z-transformacija funkcije $H(z)$.

5.2 Tonovi u praznom hodu

Važan problem koji je potrebno pomenuti važi za sve realizacije modulatora. Idle ton je pojava tonova koja se pojavljuje usled periodičnosti izlaznih vrednosti modulatora, neizbežna je i nužna. Za modulator prvog reda, iz na primer (3.5), sa jednobitnim kvantizatorom važi:

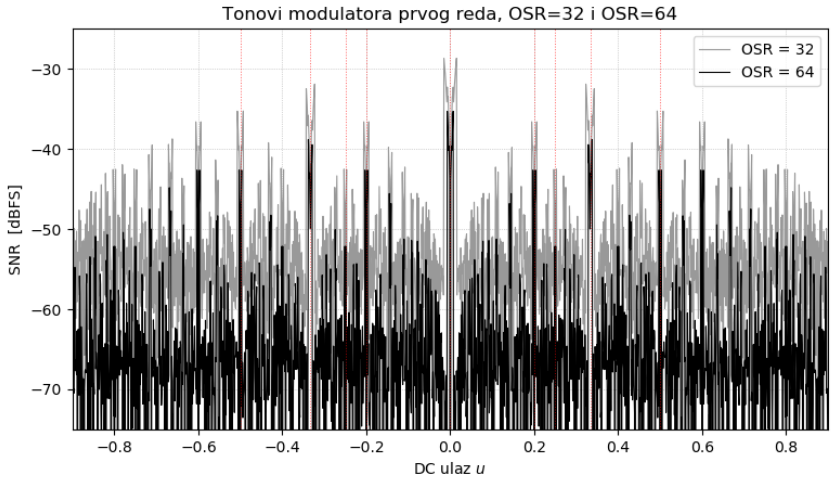
$$\begin{aligned} y(n) &= u(n) + y(n-1) - v(n-1) \\ v(n) &= sgn(y(n)) \\ &\Rightarrow \\ y(n) &= u(n) + y(n-1) - sgn(y(n-1)) \end{aligned} \tag{5.6}$$

Ako pretpostavimo da je ulazni signal $u = 1/2$ i da je početna vrednost $y(0) = 0$, možemo izvesti sledeću tabelu koja prikazuje ponašanje modulatora prvog reda.

n	**0**	**1**	**2**	**3**	**4**
$y(n)$	$\frac{1}{2}$	0	$-\frac{1}{2}$	1	$\frac{1}{2}$
$v(n)$	1	1	-1	1	1

Tabela 5.1: Tabela ponašanja modulatora prvog reda za $u = 1/2$

Bitno zapažanje iz tabele je da je izlaz periodičan sa periodom $T = 4$. Može se pokazati da se dobija periodičnost za sve se racionalne vrednosti ulaza $u = a/b$, samo sa drugačijom periodom [5, str. 43]. Periodičnost signala u spektru se odražava kao dodatna komponenta, kao ton, i to je upravo ton u praznom hodu (*engl. Idle Tone*). Grafički prikazan efekat tona u praznom hodu je prikazan na slici 5.2, sa kog možemo da vidimo da se najviši tonovi pojavljuju oko racionalnih vrednosti $u = 0, \pm 1/2, \pm 1/3 \ldots$.



Slika 5.2: Tonovi u praznom hodu u spektru

5.3 Problemi u analognim modulatorima

U analognoj realizaciji modulatora, zbog neidealnosti u proizvodnji pojačavača može doći do dodatnih problema. Missmatch pojačavača može dovesti do nestabilnosti sistema [5]. Neidealnost pojačavača, odnosno konačno pojačanje pojačavača može dovesti do degradacije $SQNR$ [5, str. 50]. Jitter signala takta unosi dodatni šum u sistem koji se ne može eliminisati [6, str. 152].

Pošto nije tema rada, probleme u analognim modulatorima ćemo samo pomenuti ali ne i obraditi. Potrebno je napomenuti da projektovanje $\Delta\Sigma$ konvertora unosi dodatne probleme ali i njihova rešenja: Neidealnost DAC [5, str. 186, 200, 243] dodatno degradiraju $SQNR$ i mogu dovesti do nestabilnosti sistema, a zanimljivo rešenje je zamena DAC sa RAM memorijom koja ima *hard code*-ovane izlaze za sve moguće ulazne vrednosti [5, str. 236].

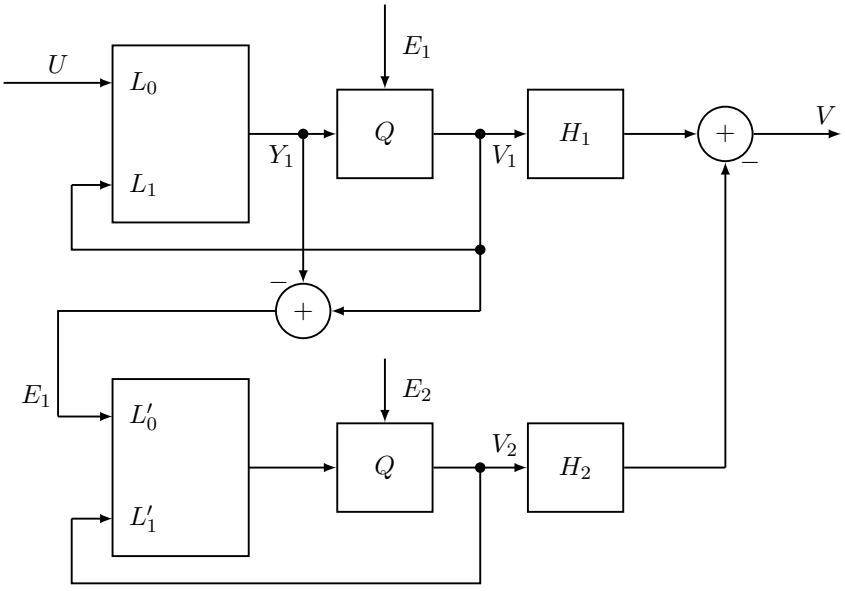
Glava 6

Strukture filtra petlje

Nakon što smo isprojektovali željene karakteristike sistema, NTF, red filtra petlje, OSR, broj kvantizacionih nivoa M, prelazimo na korak odabira strukture kojom ćemo realizovati filter. Vodeči se zaključkom iz 4.2, promenom strukture filtra ne možemo poboljšati $SQNR$ u opsegu, čim fiksiramo NTF, fiksiramo i $SQNR$. Na izbor imamo veliki dijapazon i izbor nam određuju sada drugi parametri. Treba napomenuti da sistem koji je nestabilan u jednoj strukturi, ne mora biti nestabilan i u drugoj, tako da problem stabilnosti možemo rešiti promenom strukture.

6.1 Kaskadiranje (MASH)

Pre nego spomenemo osnovne strukture filtra petlje, vredi napomenuti da je moguće kaskadirati više modulatora kao što je prikazano na slici 6.1. Kaskadiranjem modulatora, u kontekstu se z ove $MASH$ struktura. $MASH$ strukturom sastavljenom od modulatora drugog reda, dobijamo poboljšanje $SQNR$ kao da smo dizajnirali modulator četvrtog reda, ali sa prednostima stabilnosti drugog reda.



Slika 6.1: Dvostepena $MASH$ struktura, slika preuzeta iz [5]

Možemo kaskadirati i više modulatora, dokle god smatramo da je vredno kaskadirati. Cena kaskadiranja je povećanje kompleksnosti, broja komponenti i najbitnije povećanje kvantizatora. U digitalnom domenu to nije problem, ali za analogne implementacije vredno je uzeti u obzir i ove faktore. Za $MASH$ strukturu drugog reda, važe sledeće prenosne funkcije na izlazima:

$$
\begin{aligned}
V_1(z) &= STF_1(z)U(z) + NTF_1(z)E_1(z) \\
V_2(z) &= STF_2(z)E_1(z) + NTF_2(z)E_2(z) \\
\Rightarrow V &= [H_1 STF_1]U + [H_1 NTF_1 - H_2 STF_2]\, E_1 + [H_2 NTF_2]\, E_2
\end{aligned} \tag{6.1}
$$

Filtere H_1 i H_2 projektujemo tako da kvantizacioni šum prvog stepena anuliramo, i to na sledeći način:

$$
H_1 NTF_1 - H_2 STF_2 = 0 \tag{6.2}
$$

U najprostijem slučaju, projektujemo $H_1 = STF_2$ i $H_2 = NTF_1$. Zamenom ovog uslova nazad u jednačinu (6.1) teoretski ubijamo šum prvog stepena, i ovakvo rešenje prikazano je u jednačini (6.3). Naravno ukoliko nisu savršeno spregnuti filteri H_1 i H_2 pojaviće se degradacija $SQNR$ u opsegu.

$$
V = STF_1 STF_2 U - NTF_1 NTF_2 E_2 \tag{6.3}
$$

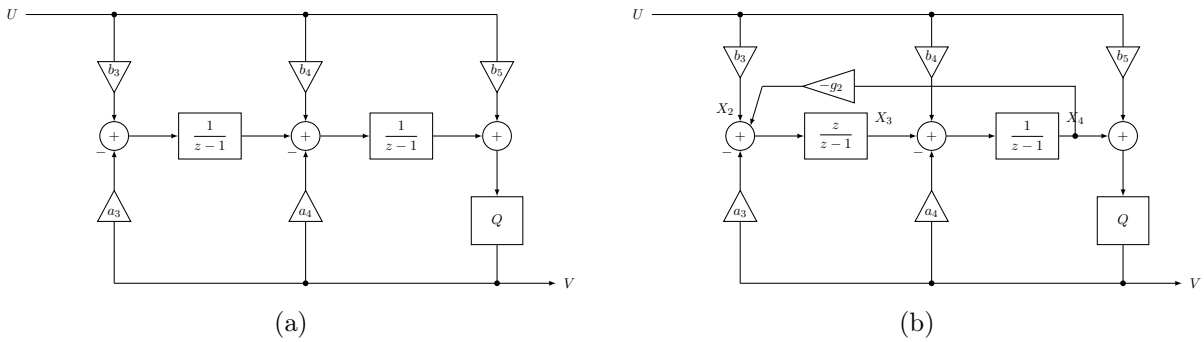
6.2 Osnovne strukture filtra petlje

Osnovne strukture u realizaciji filtra petlje su: $CIFF, CIFB$,$CRFF, CRFB$. Najlakše ih je podeliti u tabelu radi boljeg uvida u njihove povezanosti.

	Feedback	**Feed forward**
Cascaded Integrators	CIFB	CIFF
Cascaded Resonators	CRFB	CRFF

Tabela 6.1: Tabela ponašanja modulatora prvog reda za $u = 1/2$

$CIFB$(Cascaded Integrators with distributed Feedback) i $CRFB$(Cascaded Resonators with distributed Feedback) su realizacije sa feedback strukturom, odnosno rezultat kvantizacije vraćamo nazad povratnom spregom u kolo, kao što smo obrađivali do sada. Nedostatak $CIFB$ strukture se ogleda u tome da nule ne možemo proizvoljno da pomeramo po jediničnom krugu i fiksirane su na $z = 1$. Modifikacija ove strukture je $CRFB$ koja u svojoj mreži implementira rezonatore umesto integratora, i ona upravo omogućava slobodno pomeranje nula po jediničnom krugu. Primeri ovakvih struktura drugog reda prikazani su na slici 6.2.



Slika 6.2: Struktura sa raspodeljenom povratnom spregom a) $CIFB$ b) $CRFB$

Parametri filtera petlje za $CIFB$ strukturu N-tog reda u opštem slučaju daju sledeće vrednosti:

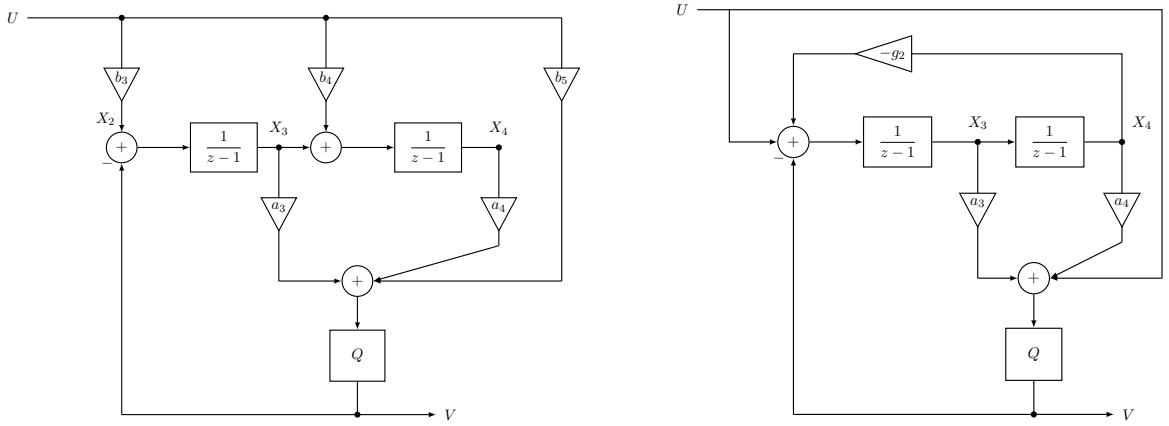
$$L_0(z) = \sum_{i=1}^{N+1} \frac{b_i}{(z-1)^{N+1-i}} = \frac{b_1 + b_2(z-1) + \cdots + b_{N+1}(z-1)^N}{(z-1)^N}$$
$$L_1(z) = \sum_{i=1}^{N} \frac{-a_i}{(z-1)^{N+1-i}} = -\frac{a_1 + a_2(z-1) + \cdots + a_N(z-1)^{N-1}}{(z-1)^N}, \qquad a_1, b_1 > 0 \tag{6.4}$$

Po ovoj notaciji, NTF i STF dati su sledećim izrazima:

$$NTF = \frac{(z-1)^N}{D(z)}$$
$$STF = \frac{b_1 + b_2(z-1) + \cdots + b_{N+1}(z-1)^N}{D(z)} \tag{6.5}$$
$$D(z) = a_1 + a_2(z-1) + \cdots + a_N(z-1)^{N-1} + (z-1)^N$$

Pronicljiv način na se izbegne problem sa stabilnošću je da se izabere vrednost koeficijenta u (6.5), $b_{N+1} = 1$ a sve ostale vrednosti koeficijenata a_i i b_i a svako $i < N$ se biraju da budu jednake [5, str. 117]. Ovakvom postavkom koeficijenata garantuje se $STF = 1$ i ne unosimo distorziju u signal. Sistem sa ovakvim koeficijentima se zove **low distorsion** modifikacija strukture.

Sa druge strane, tu su i strukture sa feed forward strukturom, $CIFF$(Cascaded Integrators with Feed Forward) i $CRFF$(Cascaded Resonators with Feed Forward). Ovakav vid realizacije filtra petlje koristi manje sabirača, tačno jedan, ali taj jedan mora da ima podršku sabiranja više signala, čime se kompleksnost sistema nije smanjila. Prednost ovakve strukture je što se izlaz modulatora ne vraća u povratnu spregu, i sistem dolazi do ustaljenja ranije. Isto kao i kod strukura pomenutih ranije, moguće je realizovati feed forward strukture sa integratorima i rezonatorima. Primeri realizacije drugog reda su prikazani na slici 6.3.



(a) (b)

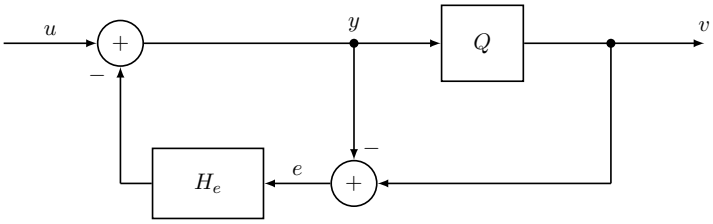
Slika 6.3: Struktura sa prenosom u napred a) $CIFF$ b) $CRFF$

$$L_1(z) = -a_1 I(z) - a_2 I^2(z) - \cdots - a_N I^N(z)$$
$$L_0(z) = b_1\big(a_1 I + a_2 I^2 + \cdots + a_N I^N\big) + b_2\big(a_2 I + \cdots + a_N I^{N-1}\big) + b_3\big(a_3 I + \cdots + a_N I^{N-2}\big) + \cdots + b_{N+1} \tag{6.6}$$

Isto kao za feedback strukture, i u feed forward strukturama moguće je napraviti low distorsion modifikaciju na isti način.

6.3 Error feedback

Struktura koja ima jako velike prednosti u digitalnom domenu, ali nepraktična za realizaciju u analognom, je *Error Feedback* struktura sa slike 6.4. Praktičnost, odnosno nepraktičnost leži u tome što u povratnoj sprezi se vraća greška $e = w - v$, a ne rezultat kvantizacije. U digitalnom domenu, vraćanje greške znači veća stabilnost i potrebno je manje bita za prenos ili za registar, dok u analognom domenu neidealnosti analognog oduzimača čine ovu strukturu praktično neupotrebljivom.



Slika 6.4: Error Feedback struktura

Prenosna funkcija sistema na izlazu data je izrazom:

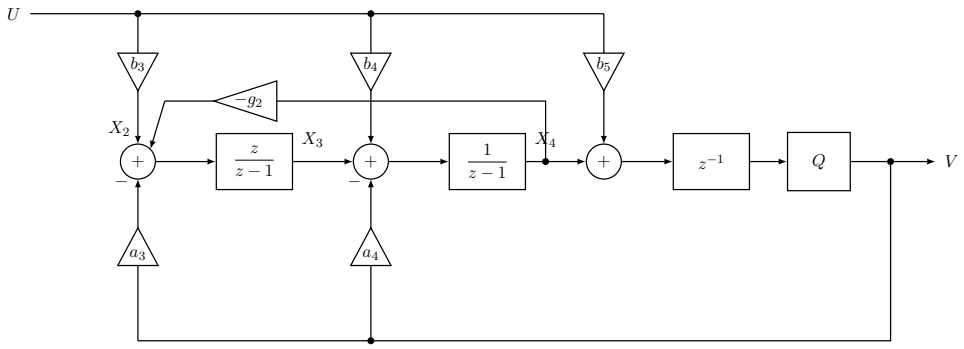
$$V(z) = U(z) + (1 - H_e(z))E(z) \tag{6.7}$$

Vidimo da je $STF = 1$ i da ne slabimo signal što je još jedan od benefita ove strukture. Filtar u povratnoj sprezi $H_e(z)$ je direktno određen NTF funkcijom i za prvi red NTF on je samo jedno kašnjenje tj. u digitalnom domenu samo registar. Implementacija za više redove je takođe jednostavna doduše pojavljuje se problem sa stabilnošću, pošto za velike vrednosti ulaznog signala, vrednost pre nego što uđe u kvantizator može porasti dovoljno da saturiše kvantizator i onesposobi sistem. U ove svrhe, potrebno je dodati digitalni limiter koji će ograničiti ulaz u kvantizator.

Posebna prednost koja se ogleda u digitalnom domenu je što sabirač u povratnoj sprezi, koji računa grešku e koja se šalje na ulaz, je ništa drugo nego ostatak kvantizatora. A kako kvantizator u digitalnom domenu predstavlja odsecanje nižih LSB bita, tako i ovaj sabirač upravo te odsečene bite vraća na ulaz, koje ne unosi značajno kašnjenje.

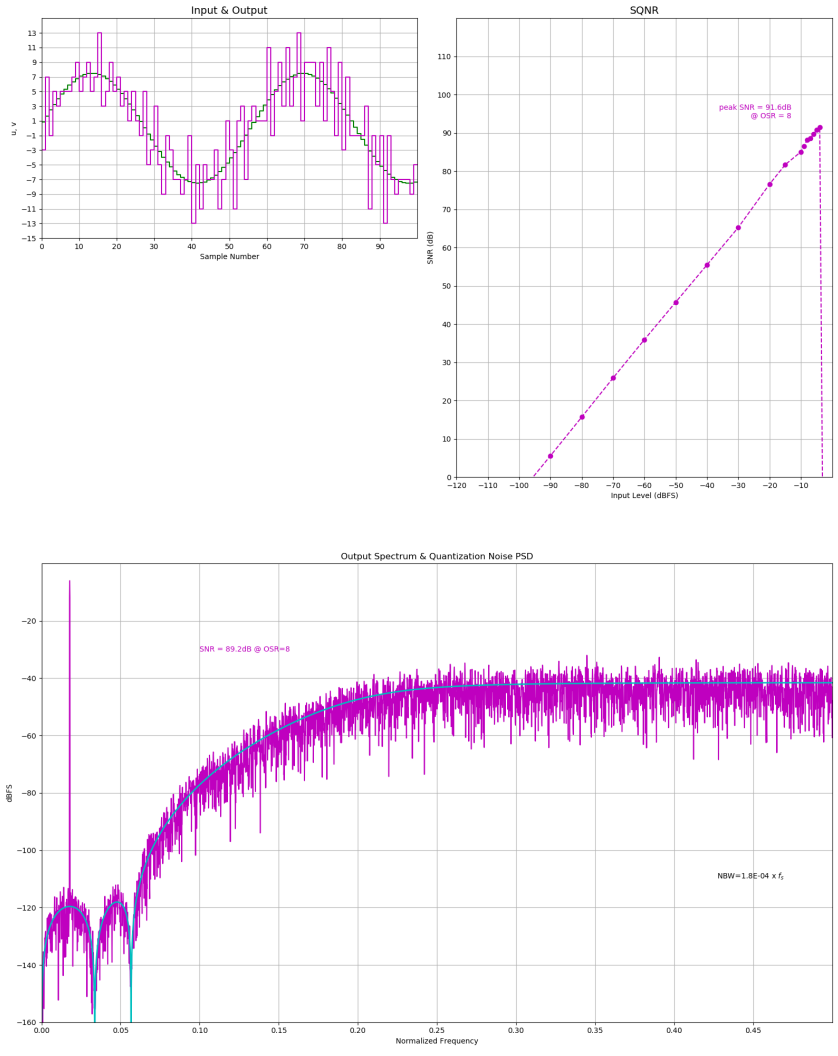
6.4 Projektovanje CRFBD

Za primer projektovanja modulatora uzećemo modifikovanu $CRFB$ strukturu, $CRFBD$ (Cascaded Resonators with Distributed Feedback and Delay) sa slike 6.5. Modifikacija se vrši dodavanjem jednog bloka za kašnjenje pre kvantizatora. Na taj način skraćujemo dužinu kritične putanje čime dobijamo na taktu, a i usputno dobijamo na stabilnosti pošto rasterećujemo kvantizator. Cena koju plaćamo je da $STF_{CRFBD} = z^{-1}STF_{CRFB}$, što je nikakva cena za našu karakteristiku i performanse. Kod korišćen u projektovanju modulatora, pisan u python-u, nalazi se u dodatku A.1.

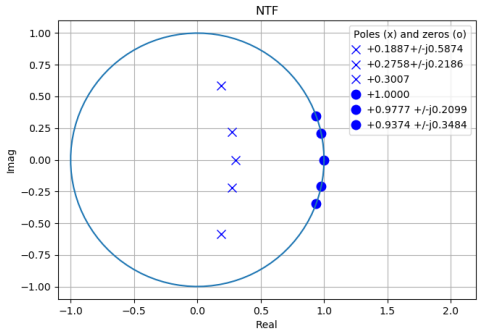


Slika 6.5: CRFBD struktura

Modulator ćemo projektovati za $OSR = 8$ i kvantizator sa $M = 15$ nivoa, odnosno 4-bitni kvantizator sa filterom u povratnoj sprezi reda $N = 5$. Simuliranjem u python-u, dobijamo grafike prikazane na slici 6.6 i rezultujući $SQNR$ iznosi $89.2dB$.

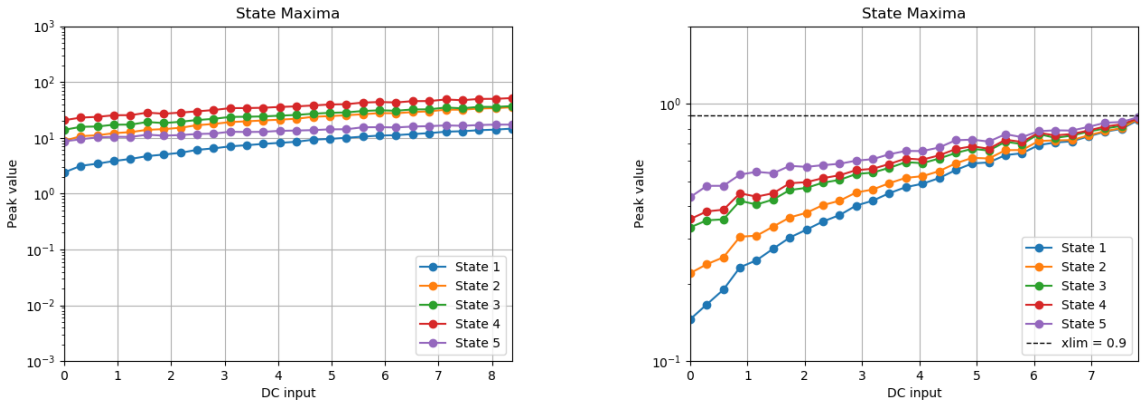


Slika 6.6: SQNR grafici projektovanog modulatora



Slika 6.7: Položaji polova i nula projektovanog modulatora

Na slici 6.7 vidimo kako izgledaju položaji polova i nula projektovanog NTF. Pošto smo odredili NTF, možemo odrediti vrednosti $ABCD$ za datu strukturu. Pošto su pojačanja signala između redova strukture neujednačena, kako se i vidi sa slike 6.8a, implementiranom funkcijom `scaleABCD`, optimizujemo pojačanja između redova strukture.. Rezultat skaliranja možemo videti na slici 6.8b.



(a) (b)

Slika 6.8: Pojačanja između redova strukture a) pre optimizacije b) posle optimizacije

Vrednosti koeficijenata dobijeni iz skalirane $ABCD$ matrice naše strukture prikazani su u okviru tabele 6.2. Rešenja dobijena projektovanjem dalje prosleđujemo u C++ gde ćemo preciznije ispitati ponašanje našeg sistema.

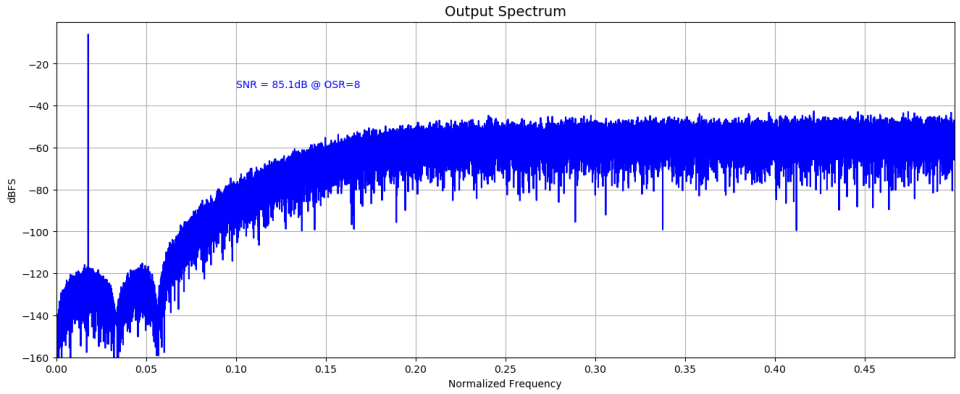
i	a_i	b_i	c_i	g_i
1	0,0242	0,0242	0,4116	0,0466
2	0,0327	0	0,9569	0,0431
3	0,0743	0	0,7231	—
4	0,0450	0	2,9072	—
5	0,1800	0	20,0005	—

Tabela 6.2: Vrednosti koeficijenata iz $ABCD$ matrice

6.5 Simulacije CRFBD

Simulacije vršimo u C++ programskom jeziku pomoću paketa *ac_types* [12]. Vrednosti dobijene projektovanjem nećemo direktno ubaciti u simulaciju. Pošto je u hardveru množenje skupa operacija i ima veliko kašnjenje, izbegavaćemo je već ćemo množenje realizovati kao pomeranje i sabiranje. Adekvatne realizacije ovakvog množenja i kompletan kod korišćen u simulacijama nalaze se u dodatku A.2.

Rezultat simulacija je grafik prikazan na slici 6.9. Na slici možemo videti da je $SQNR$ u opsegu oslabljen za $4dB$ u odnosu na simulacije iz python-a. Razlog ovoga je što množače aproksimiramo pomeračima radi uštede na hardveru i to je cena koju plaćamo.



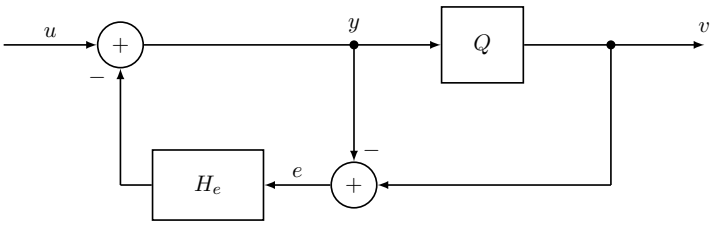
Slika 6.9: Rezultat simulacije u C++

Glava 7

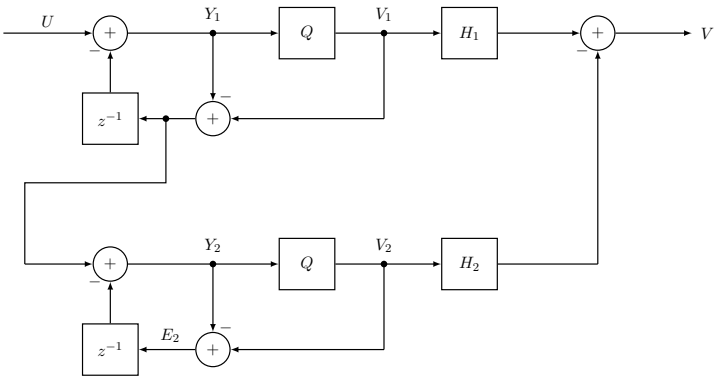
Postupak projektovanja višebitnog MASH(1-1) Error Feedback modulatora

Iako rezultati simulacija modulatora iz odeljka 6.5 poseduju zadovoljavajuć nivo kvantizacionog šuma, takav modulator je skup i nepraktičan za izradu u digitalnom hardveru. Bolja struktura za realizaciju u digitalnom hardveru je error feedback struktura pomenuta u odeljku 6.3. Ako bismo projektovali error feedback strukturu na osnovu NTF projektovanog u odeljku 6.4, rezultujući modulator bi praktično bio neupotrebljiv na visokim učestanostima zbog veoma velike kritične putanje [13, str. 44]. Iako je moguće skratiti dužinu kritične putanje umetanjem dodatnih registara u kritičnu putanju, time bismo smanjili $SQNR$ u opsegu i degradirali performanse modulatora, kao i zakomplikovali strukturu.

U ovom radu, kao rešenje ovih problema, predložena je MASH(1-1) error feedback struktura pomenuta u radu [14]. Počećemo od kaskadiranja dva modulatora prvog reda, error feedback strukture kao na slici 7.2.



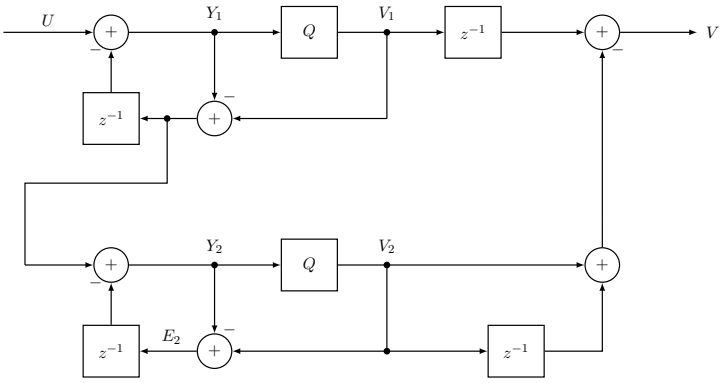
Slika 7.1: Error Feedback struktura, ponovljena slika iz odeljka 6.3



Slika 7.2: $MASH$(1-1) Error Feedback struktura

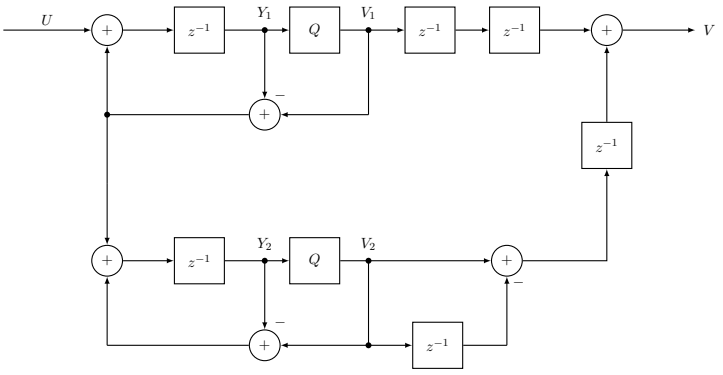
Kako prenosna funkcija greške modulatora prvog reda iznosi $NTF = 1 - z^{-1}$, filteri $H_e = 1 - NTF$ u naš sistem unose samo zakašnjenje. Filter u povratnoj sprezi H_e u naš sistem ne unosi nikakve dodate sabirače ni množače, a samim tim ima malo kašnjenje. Filtere na izlazu stepena H_1 i H_2 projektujemo po zaključku iz jednačine (6.2) tako da se greška prvog stepena anulira. Rezultujuća jednačina na izlazu MASH(1-1) error feedback strukture je data izrazom:

$$V(z) = U(z) - (1 - z^{-1})E_2(z) \tag{7.1}$$



Slika 7.3: Kompletna MASH(1-1) Error Feedback struktura

Međutim, neoptimizovana struktura kakva je prikazana na slici, i dalje ima veliku dužinu kritične putanje. Brzom analizom slike 7.3 možemo uočiti da je najduža kritična putanja sastavljena od 2 sabirača, između izlaza drugog stepena na izlaz sistema, izlaza prvog stepena na izlaz sistema i između prvog i drugog stepena $MASH$ strukture. Problem možemo rešiti dodavanjem registara između sabirača, ali možemo se poslužiti i još jednom forom. Prebacivanjem registara iz povratne sprege u jednom stepenu strukture, u granu signala NTF tog stepena ne menjamo, menjamo samo STF. Gubimo bezuslovnu stabilnost ali šta dobijamo? Dobijamo da smo prekinuli kritičnu putanju između 2 stepena strukture. Sabiraču na izlazu ćemo dodati registar po grani kako bismo i tu prekinuli kritične putanje. Optimizovana struktura prikazana je na slici 7.4.



Slika 7.4: Optimizovana MASH(1-1) Error Feedback struktura

Prenosna karakteristika ovakvog sistema data je sledećim izrazom:

$$\begin{aligned} V_i(z) &= z^{-1}U(z) - (1 - z^{-1})E_i(z) \qquad i = 1, 2 \\ V(z) &= z^{-3}U(z) - z^{-1}(1 - z^{-1})^2 E_2(z) \end{aligned} \tag{7.2}$$

7.1 Projektovanje NTF

Prvi korak u projektovanju je provera koliko kvantizacionog šuma ovakva realizacija suzbija. Proveru vršimo u python-u simuliranjem NTF prvog reda, potom računanjem njegove $ABCD_i$ matrice za proizvoljnu strukturu i na kraju spajanjem dveju matrica u jednu $ABCD$ matricu. Kodovi korišćeni za projektovanje dati su u dodatku B.3.

$ABCD_i$ matricu ne možemo dobiti pomoću ugrađene funkcije `stuffABCD` jer ona ne podržava NTF prvog reda, tako da ćemo ručno izračunati $ABCD_i$ matricu za NTF prvog reda.

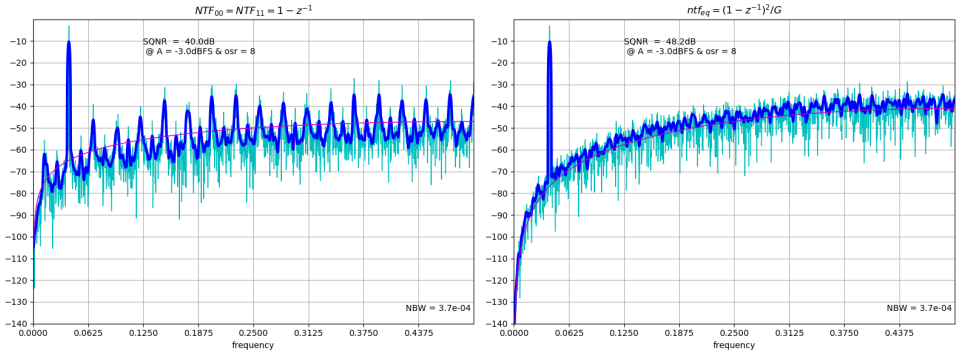
$$\mathrm{ABCD_i} = \left[\begin{array}{c|cc} A & B_u & B_v \\ \hline C & D_u & D_v \end{array}\right] = \left[\begin{array}{c|cc} 1 & 1 & -1 \\ \hline 1 & 0 & 0 \end{array}\right] \tag{7.3}$$

Kaskadiranu matricu $ABCD$ dobijamo direktnim zbirom dveju linearnih prostora, u čije objašnjavanje nećemo zalaziti jer je van opsega rada, ali generalizovano objašnjenje se može naći u literaturi [9, str. 110]. Kaskadiranu matricu mapiramo na sledeći način, u kojoj je G pojačanje između stepena:

Mapiranje kaskadirane $ABCD$ matrice

$$\mathrm{ABCD} = \left[\begin{array}{cc|cc|c} A_1 & 0 & B_{u1} & B_{v1} & 0 \\ -G\,B_{u2} & A_2 & 0 & 0 & B_{v2} \\ \hline C_1 & 0 & D_{u1} & D_{v1} & 0 \\ -G\,D_{u2} & C_2 & 0 & 0 & D_{v2} \end{array}\right]$$

Simulacijom dobijamo sledeći grafik. Na grafiku su upoređeni NTF celog sistema i NTF_{11} koji predstavlja samo NTF drugog stepena. Možemo videti da je suzbijanje greške kvantizacije sa prvog stepena veoma efektivno i da unosi samo $8dB$ šuma.

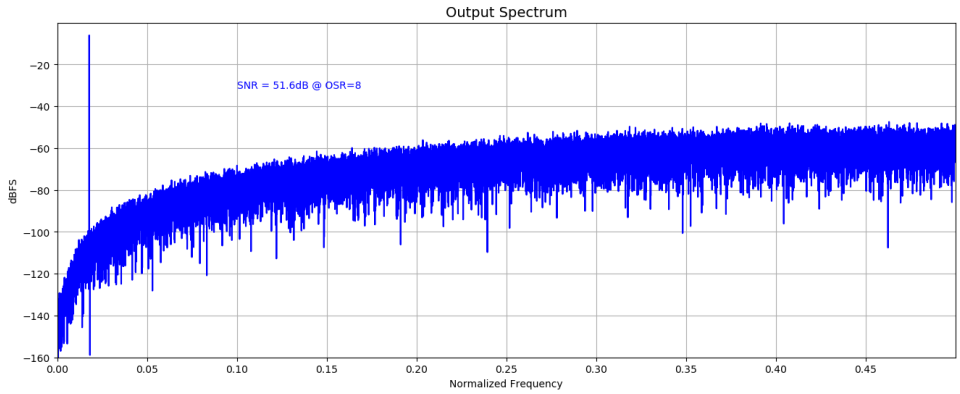


Slika 7.5: Rezultat simulacije u python-u

$SQNR$ dobijen ovakvim simulacijama sa grafika 7.5 iznosi $SQNR = 48.2dB$, znatno niže nego simulacije za $CRFBD$ strukturu ali to je očekivano jer je red modulatora mnogo manji.

7.2 Simulacije u C++

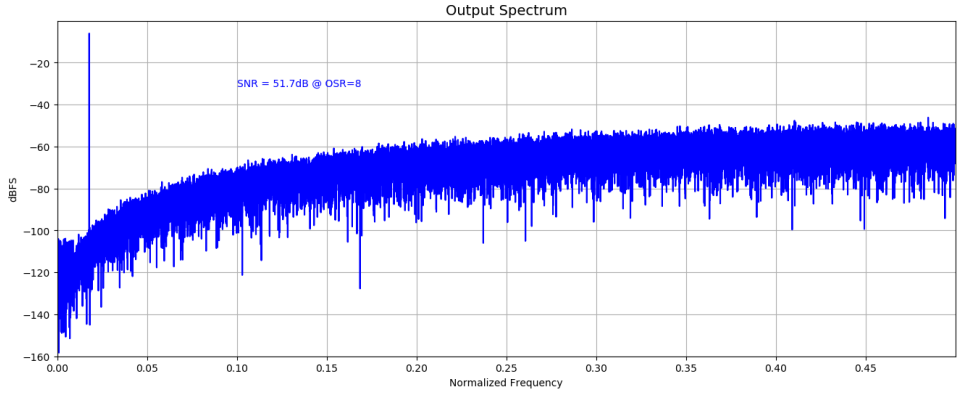
Postupak simuliranja u C++ je praktično isti kao i u odeljku 6.5, doduše lakše jer nemamo množače u sistemu. Rezultat simulacija je prikazan na slici 7.6. Kompletan kod je datu u dodatku B.2.



Slika 7.6: Rezultat simulacije u C++

7.3 Simulacije u VHDL

Simulacije u VHDL su vršene u fiksnom zarezu uz pomoć biblioteke `fixed_pkg`. Kompletan kod je datu u dodatku B.3, a rezultati su prikazani na slici 7.7. Rezultati su u skladu sa simulacijama u C++ i python-u, što je očekivano.



Slika 7.7: Rezultat simulacije u VHDL-u

7.4 Delta-sigma na višim učestanostima

Projektovanje i simuliranje $\Delta\Sigma$ modulatora samo po sebi ne predstavlja veliki izazov, pravi izazov je projektovati modulator koji podržava rad na visokim učestanostima. U ovom odeljku biće kratko teoretski obrađeno koja su ograničenja ako bismo projektovali takav modulator.

Za primer uzećemo slične parametre kao u radu [14]. Ako razmatramo mogućnosti konstruisanja modulatora za prijemnik koji implementira $WIFI$ 6E standard, potrebno je da modulator može da obradi signale sa centralnom učestanošću na $6GHz$ sa širinom propusnog opsega $BW = 80MHz$ u najgorem slučaju.

Kako bismo zadovoljili Nikvistov kriterijum odabiranja moramo odabirati sa minimalnom učestanošću $f_s = 2BW = 160MHz$. Koristeći formule iz odeljka 2.6, vidimo da je maksimalna Nikvistova zona u kojoj možemo odabirati $k = 75$. Pošto nam je najzgodnije, želimo da odabiramo tako da se centralna učestanost signala preslika u $f_s/4$ normalizovanog opsega tako da ćemo izabrati 9. Nikvistovu zonu. Prema formulama iz istog odeljka dobijamo da je minimalna učestanost odabiranja u 9. Nikvistovoj zoni $f_s = 1.34GHz$ za koji nam je potreban $OSR = 8.84$. OSR kojim smo projektovali sistem je sasvim zadovoljavajuć, u normalizovanom spektru učestanosti naš signal će biti samo pomeren desno. Uz pretpostavku, da je radna učestanost sabirača

viša nego učestanost takta, ovakav sistem je sasvim moguće ostvariti u digitalnom hardveru.

Nedostatak projektovanog modulatora leži u tome što se koristan signal ne preslikava blizu dc, tako da low pass realizacija modulatora nije od koristi. Neophodno je ili projektovati bandpass ili kvadraturni modulator. Kvadraturni modulator je u ovom slučaju bolja opcija jer je modulator efikasniji. Ove realizacije u radu nisu obrađene jer su malo zahtevnije i ne bi mogle biti realizovane sa ovako malom dužinom kritične putanje.

Kako naš sistem ima $SQNR = 51dB$, SNR_j koje pravi šum podrhtavanja trenutka odabiranja ćemo projektovati tako da nam ne remeti ukupan SNR kola, odnosno na $SNR_j = 57dB$. Koristeći se jednačinom (2.18), brzom smenom vrednosti parametara dobijamo da je podrhtavanje trenutka odabiranja $t_j = 37.5fs$. Ovakvi zahtevi su prilično nezahvalan oscilatori sa tom preciznošću su prilično skupi.

Glava 8

Zaključak

U ovom radu je u potpunosti obrađen put projektovanja višebitnog $\Delta\Sigma$ modulatora. Iako grdna teorija koja je uvedena u projektovanju nema veliku ulogu, mnogo je bitnije razumeti kako se koristi modulator i koje su njegove mogućnosti od samog projektovanja.

Modulator koji je projektovan na kraju najpre je izabran zbog njegove jednostavnosti projektovanja i simuliranja i zbog zanimljive kombinacije $MASH$ i *Error Feedback* strukture. Rezultati dobijeni u [14] su takođe privukli autora da se pozabavi baš ovom arhitekturom.

Performanse modulatora koji je na kraju projektovan, iako manje zadovoljavajuće od performansi modulatora iz [13], rešava problem dužine kritične putanje i omogućava rad na visokim učestanostima. Na kraju, teoretska obrada mogućnosti ovog modulatora pokazuje nam da je moguć rad na visokim učestanostima, u teoriji. Usko grlo je izbor oscilatora sa velikom preciznošću, ali i taj problem je rešiv promenom predložene strukture. Ukoliko bi se implementirao kvadraturni modulator, umesto linearnog, obezbedili bismo bolje oblikovanje kvantizacionog šuma i relaksirali zahteve na preciznost oscilatora. Moguća je još jedna dodatna modifikacija strukture koja uključuje polifazno razlaganje kola. Ovakav vid modifikacije je dosta napredan i samo simulacije za takav sistem nisu dovoljne.

Dodavanje redova u strukturi bi takođe bila opcija u cilju poboljšanja performansi modulatora, ali onda bismo se vratili na problem dužine kritične putanje. Ali naravno, dodatnim optimizovanjem strukture, ubacivanjem registara u kritičnu putanju mogli bismo da ostvarimo zahtev rada na visokim učestanostima, i tako nazad povratnom spregom u krug.

Nažalost, hardverska realizacija modulatora nije bila ostvarena i projektovanje modulatora se završava samo na simulacijama i spekuliranju, ali rezultati dobijeni simulacijama se ne bi razlikovali mnogo od sintetisanog modulatora.

Veštačka inteligencija jeste bila korišćena pri izradi ovog rada. Njena funkcija je bila svedena na alatku, kako je i zamišljena da se koristi. Ispravljanje gramatičkih grešaka, grešaka u kodovima, sanity check, crtanje većine grafika, pisanje jednog dela jednačina, brza provera rezultata kao i brze konsultacije su bili poslovi koje veštačka inteligencija obavljala kako bi ubrzala i ulepšala ovaj rad. Sve rečenice, ideje i zaključci su potpuno originalni i ljudski.

Bibliografija

[1] dr. Miodrag V. Popović, *Digitalna Obrada Signala*, Nauka, Beograd, 1996.

[2] dr. Dušan N. Grujić, *Hardversko Softverska Obrada Signala*, Univerzitet u Beogradu Elektrotehnički fakultet, Beograd, 2025, *preprint*.

[3] Behzad Razavi, *The Delta-Sigma Modulator*, IEEE Solid-State Circuits Magazine, 2016.

[4] H. Inose, Y. Yasuda, J. Murakami, *A telemetering system by code modulation- Delta Sigma modulation*, IEEE Transactions on Space Electronics and Telemetry, 1962.

[5] Richard Schreier, Gabor C. Temes, *Understanding Delta-Sigma Data Converters*, IEEE Press, USA, 2017.

[6] George I Bourdopoulos, *Delta-Sigma Modulators - Modeling, Design and Applications*, Imperial College Press, London, 2003.

[7] dr. Predrag Pejović, *Algebarska degeneracija u električnim kolima*, Dopunski materijal sa časova Energetske elektronike, 2023.

[8] Vishal Saxena, *Higher-order Delta-Sigma Modulators and ABCD Matrix Modeling*, Boise State University, 2010.

[9] Chi-Tsong Chen, *Linear Systems Theory and Design*, State University of New York at Stony Brook, College Publishing, 1984.

[10] W. L. LEE, *A novel higher order interpolative modulator topology for high resolution oversampling A/D converters*, Massachusetts Institute of Technology, Cambridge, MA, 1987.

[11] Giuseppe Venturini, *https://www.python-deltasigma.io/*, 2016.

[12] Andres Takach, *https://github.com/hlslibs/ac_types*, 2026.

[13] Luka Bogojević, *Digitalni Višebitni Delta-Sigma Modulatori*, Diplomski rad, Katedra za elektroniku i digitalne sisteme, Univerzitet u Beogradu Elektrotehnički fakultet, Beograd, 2023.

[14] Albert Jerng, *Delta-Sigma Digital-RF Modulation for High Data Rate Transmitters*, Ph.D. dissertation, Department of Electrical and Computer Science, Massachusetts Institute of Technology, Cambridge, MA, 2007.

[15] GHDL, *https://github.com/ghdl/ghdl*.

[16] GTKWave, *https://gtkwave.sourceforge.net/*.

Dodatak A

Projektovanje i simulacije CRFBD modulatora

U ovom dodatku nalaze se kodovi korišćeni u projektovanju i simuliranju modulatora sa $CRFBD$ strukturom. Kako bi se veličina ovog rada smanjila, izbačene su uvodne linije koda za importovanje bibliotekao kao i funkcije za plotovanje grafika i rezultata u python-u. Projektovanje je vršeno u jupyter sveskama uz podršku biblioteke za projektovanje $\Delta\Sigma$ modulatora. Sveske i paket mo moguće je preuzeti sa [11]. Ove python sveske su kopija MATLAB kodova iz literature [5]. Simuliranje vršeno je u C++ programskom jeziku, sa podrškom za fiksni zarez iz biblioteke ac_types koju je moguće preuzeti sa [12].

A.1 Projetovanje modulatora

Prvo iterativno simuliramo modulator za različite vrednosti $Hinf$ kako bi pronašli najvišu vrednost $Hinf$ za koju modulator ima najviše slabljenje $SQNR$ a ostaje stabilan.

Isečak koda A.1: Projektovanje modulatora.

```
order = 5
OSR = 8
M = 15
form = 'CRFBD'
opt=1
xlim = 0.9
nlev  = M + 1
N_sim = 10000

Hinf_list = [2.0, 4.0, 6.0, 7.0, 8.0, 9.0, 10.0, 12.0]
rows = []

for Hinf in Hinf_list:
    r = {'Hinf': Hinf}

    H = synthesizeNTF(order, OSR, opt, Hinf)

    snr, amp = simulateSNR(H, OSR, None, 0., nlev)
    r['pkSQNR'], r['pkAmp'] = peakSNR(snr, amp)

    rows.append(r)
```

Maksimalno slabljenje dobijamo za vrednost $Hinf = 8.0$. Posle toga pronalazimo vrednosti koeficijenata a_i, b_i, g_i, c_i.

Isečak koda A.2: Projektovanje modulatora

```
a, g, b, c = realizeNTF(H, form=form)
b = np.concatenate((b[0].reshape((1,)), np.zeros((b.shape[0] - 1,))), axis
    =0)
```

Potom konstruišemo $ABCD$ matricu, skaliramo je i mapiramo nove vrednosti koeficijenata a_i, b_i, g_i, c_i.

Isečak koda A.3: Projektovanje modulatora

```
ABCD = stuffABCD(a, g, b, c, form=form);

ABCDs, umax, S = scaleABCD(ABCD, nlev=nlev, N_sim=10000, xlim=0.9)

as_, gs, bs, cs = mapABCD(ABCDs, form=form)
```

Simuliranje ponašanja ovake ABCD matrice možemo uraditi ovako u python-u:

Isečak koda A.4: Projektovanje modulatora

```
u = np.linspace(0, 0.6*M, 30);
N = 1e4;
test_signal = np.ones((1, int(N)))
maxima = np.zeros((order, len(u)))
for i in range(len(u)):
    ui = u[i]
    v, xn, xmax, _ = simulateDSM(ui*test_signal, ABCDs, M + 1);
    maxima[:, i] = np.squeeze(xmax)
    if any(xmax > 1e2):
        umax = ui;
        u = u[:i];
        maxima = maxima[:, :i]
        break;
```

A.2 Simulacija modulatora

Prvo generišemo sinusni test signal na osnovu signala koje smo koristili u projktovanju

Isečak koda A.5: Simulacija modulatora

```
fB    = int(np.ceil(N/(2.*OSR)))
ftest = int(np.floor(2./7*fB))

def gen_sine(amp, fname="Podaci", N=N, ftest=ftest, bits=BITS, fs_q=FS_Q):
    n   = np.arange(N)
    s = np.sin(2*np.pi*ftest*n/N)
    x = amp * fs_q * s
    np.savetxt(fname, x, fmt="%.17g")
    return x

x = gen_sine(AMP)
```

Pravimo lokalnu biblioteku za učitavanje podataka iz fajla i upis u fajl koju nazivamo DSM_IO.hpp.

Isečak koda A.6: Simulacija modulatora

```
int number_of_data(const char* input_file){
    int num{}; std::fstream my_file;
    my_file.open(input_file, std::ios::in);
    if (my_file.is_open()){ double i;
        while(my_file >> i){ num++; } my_file.close();
    }
    else std::cout << "Unable to open file" << std::endl;
    return num;
}
void load_data(const char* input_file, double data[], int size){
    std::fstream my_file;
    my_file.open(input_file, std::ios::in);
    if (my_file.is_open()){
        for(size_t i=0; i<size; i++){
            my_file >> data[i];
        }
    my_file.close();
    }
    else std::cout << "Unable to open file" << std::endl;
}
void write_data(const char* output_file, int data[], int size){
    std::fstream my_file;
    my_file.open(output_file, std::ios::out);
    if (my_file.is_open()){
        for(size_t i=0; i<size; i++ ){
        my_file << data[i] << std::endl;
    } my_file.close(); }
    else std::cout << "Unable to open file" << std::endl;
}
```

Ubacujemo vrednosti koeficijenata koje smo dobili projektovanjem, i povezujemo komponente na osnovu šeme strukture.

Isečak koda A.7: Simulacija modulatora

```
#include <iostream>
#include "DSM_IO.hpp"
#include "ac_fixed.h"

ac_fixed<5, 5, true, AC_TRN, AC_SAT> quantizer(ac_fixed<5, 5, true, AC_TRN,
     AC_SAT> in){
    ac_fixed<5, 5, true, AC_TRN, AC_SAT> out = in;
    if (in <= -15) out = -15;
    else if (in >= 15) out = 15;
    else {
        if ((in & 1) == 0) out = in + 1;
    }
    return out;
}

int main(){

    ac_fixed<22,5,true,AC_TRN,AC_SAT> x1, x1d=0.0;
    ac_fixed<22,5,true,AC_TRN,AC_SAT> x2, x2d=0.0, hx2, hx2d=0.0;
    ac_fixed<22,5,true,AC_TRN,AC_SAT> x3, x3d=0.0;
    ac_fixed<22,5,true,AC_TRN,AC_SAT> x4, x4d=0.0, hx4, hx4d=0.0;
    ac_fixed<22,5,true,AC_TRN,AC_SAT> x5, x5d=0.0;
    ac_fixed<22,5,true,AC_TRN,AC_SAT> y, vout=0;
```

```
23      ac_fixed<25,5,true,AC_TRN,AC_SAT> uin;
24      ac_fixed<25,5,true,AC_TRN,AC_SAT> fb=0.0;
25
26      int n{}; n = number_of_data("Podaci");
27
28      double* p_data{new double[n] {}};
29      int* p_q{new int[n] {}};
30      load_data("Podaci", p_data, n);
31
32      for(size_t i=0; i<n; i++){
33          uin=p_data[i];
34          fb = 0.0;
35          fb = fb - vout;
36          x1   = x1d      + ((uin+fb)>>6)+((uin+fb)>>8);
37          x2   = x2d      + hx2d;
38          x3   = x3d      + x2-(x2>>3)-(x2>>6)     + (fb>>4)-(fb>>9)+(fb>>11);
39          hx2  = -((x3>>4)-(x3>>6)+(x3>>8))      + (fb>>5)-(fb>>8)-(fb>>11)
                  + (x1>>1)-(x1>>3)+(x1>>6);
40          x4   = x4d      +    hx4d;
41          x5   = x5d      + (x4<<1)+(x4>>1)-(x4>>4)     + (fb>>3)+(fb>>6)-(fb
                >>9);
42          hx4  = -((x5>>4)-(x5>>6)+(x5>>8))       + (fb>>5)+(fb>>7)         + (
                x3)-(x3>>2)-(x3>>4);
43          y    = (x5<<4)+(x5<<3)-x5;
44
45          x1d=x1;
46          hx2d=hx2;
47          x2d=x2;
48          x3d=x3;
49          hx4d=hx4;
50          x4d=x4;
51          x5d=x5;
52          vout=quantizer(y);
53          p_q[i]=vout.to_int();
54      }
55
56      write_data("Obradjeni_real", p_q, n);
57
58      delete[] p_data;
59      p_data=nullptr;
60      delete[] p_q;
61      p_q=nullptr;
62      return 0;
63  }
```

Dodatak B

Projektovanje i simulacije MASH(1-1) Error Feedback strukture

U ovom dodatku nalaze se kodovi korišćeni u projektovanju i simuliranju modulatora sa MASH(1-1) Error Feedback strukturom. Kako bi se veličina ovog rada smanjila, izbačene su uvodne linije koda za importovanje bibliotekao kao i funkcije za plotovanje grafika i rezultata u python-u. Projektovanje je vršeno u jupyter sveskama uz podršku biblioteke za projektovanje $\Delta\Sigma$ modulatora. Sveske i paket mo moguće je preuzeti sa [11]. Ove python sveske su kopija MATLAB kodova iz literature [5]. Simuliranje vršeno je u C++ programskom jeziku, sa podrškom za fiksni zarez iz biblioteke ac_types koju je moguće preuzeti sa [12]. Simuliranje u VHDL-u vršeno je u GHDL programskom okruženju [15] uz GTKWave waveform pregledač [16].

B.1 Projektovanje modulatora

Isečak koda B.1: Projektovanje modulatora.

```
import deltasigma as ds

def zpk_multiply(a, b):
    za, pa, ka = ds._utils._get_zpk(a)
    zb, pb, kb = ds._utils._get_zpk(b)
    pa = pa.tolist() if hasattr(pa, 'tolist') else pa
    pb = pb.tolist() if hasattr(pb, 'tolist') else pb
    za = za.tolist() if hasattr(za, 'tolist') else za
    zb = zb.tolist() if hasattr(zb, 'tolist') else zb
    return ds.cancelPZ((za+zb, pa+pb, ka*kb))

G = 1.0
ABCD_mash = np.array([[1., 0., 1., -1.,  0.],
                      [ G, 1., 0.,  -G, -1.],
                      [1., 0., 0.,  0.,  0.],
                      [0., 1., 0.,  0.,  0.]])

nlev   = [15, 15]
filtM1 = [0., 1.]
filtM2 = [1./G, -1./G]
k = [1., 1.]
ntfs, stfs = calculateTF(ABCD_mash, k)
```

```
23
24 M1 = tf2zpk([1.],           [1., 0.])
25 M2 = tf2zpk([1./G, -1./G], [1., 0.])
26 ntf_eq = zpk_multiply(ntfs[1, 1], M2)
```

B.2 Simulacija modulatora u C++

Za ove simulacije takodje smo se koristili lokalno kreairanom bibliotekom DSM_IO.hpp iz dodatka A

Isečak koda B.2: Simulacija modulatora

```
1  #include <iostream>
2  #include "DSM_IO.hpp"
3  #include "ac_fixed.h"
4
5
6  ac_fixed<5, 5, true, AC_TRN, AC_SAT> quantizer(ac_fixed<5, 5, true, AC_TRN,
       AC_SAT> in){
7      ac_fixed<5, 5, true, AC_TRN, AC_SAT> out = in;
8      if (in <= -15) out = -15;
9      else if (in >= 15) out = 15;
10     else {
11         if ((in & 1) == 0) out = in + 1;
12     }
13     return out;
14 }
15
16 int main(){
17     #include <iostream>
18 #include "DSM_IO.hpp"
19 #include "ac_fixed.h"
20
21
22 ac_fixed<5, 5, true, AC_TRN, AC_SAT> quantizer(ac_fixed<5, 5, true, AC_TRN,
       AC_SAT> in){
23     ac_fixed<5, 5, true, AC_TRN, AC_SAT> out = in;
24     if (in <= -15) out = -15;
25     else if (in >= 15) out = 15;
26     else {
27         if ((in & 1) == 0) out = in + 1;
28     }
29     return out;
30 }
31
32 int main(){
33     ac_fixed<16, 5, true, AC_TRN, AC_SAT> uin;
34     ac_fixed<5, 5, true, AC_TRN, AC_SAT> vout=0;
35     ac_fixed<12, 5, true, AC_TRN, AC_SAT> w1, w1d=0;
36     ac_fixed<16, 1, true, AC_TRN, AC_SAT> e1=0;
37     ac_fixed<5, 5, true, AC_TRN, AC_SAT> y1, v1=0, y1d=0;
38     ac_fixed<16, 5, true, AC_TRN, AC_SAT> w2, w2d=0;
39     ac_fixed<12, 1, true, AC_TRN, AC_SAT> e2=0;
40     ac_fixed<5, 5, true, AC_TRN, AC_SAT> y2, y2d=0, v2=0, v2d=0;
41
42
43     int n{}; n = number_of_data("Podaci");
```

```
    double* p_data{new double[n] {}};
    int* p_q{new int[n] {}};
    load_data("Podaci", p_data, n);

    int i;
    for(size_t i=0; i<n; i++){

        uin=p_data[i];
        vout = v1 + v2d;

        y1 = quantizer(w1d);
        e1 = (w1d - y1);
        w1 = uin + e1;

        y2 = quantizer(w2d);
        e2 = (w2d - y2);
        w2 = e1 + e2;

        v2 = y2- y2d;


        v1 = y1d;
        y1d = y1;
        w1d = w1;
        w2d = w2;
        y2d = y2;
        v2d = v2;

        p_q[i]=vout.to_int();

    }
    write_data("Obradjeni_mash_real", p_q, n);
    delete[] p_data;
    p_data=nullptr;
    delete[] p_q;
    p_q=nullptr;
    return 0;
}
```

B.3 Simulacija modulatora u VHDL

Isečak koda B.3: Simulacija modulatora u VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.fixed_float_types.all;
use IEEE.fixed_pkg.all;

entity mash is
    port(
        data_in : in  sfixed(4 downto -11);
        reset, clk: in  std_logic;
        data_out: out sfixed(4 downto 0));
end mash;

architecture rtl of mash is
```

```
15      signal w1d : sfixed(4 downto  -7) := to_sfixed(0,4,-7);     -- ac_fixed
            <12,5>
16      signal w2d : sfixed(4 downto -11) := to_sfixed(0,4,-11);    -- ac_fixed
            <16,5>
17      signal y1d : sfixed(4 downto   0) := to_sfixed(0,4,0);      -- ac_fixed
            <5,5>
18      signal v1  : sfixed(4 downto   0) := to_sfixed(0,4,0);      -- ac_fixed
            <5,5>
19      signal y2d : sfixed(4 downto   0) := to_sfixed(0,4,0);      -- ac_fixed
            <5,5>
20      signal v2d : sfixed(4 downto   0) := to_sfixed(0,4,0);      -- ac_fixed
            <5,5>
21      signal vsum: sfixed(4 downto   0) := to_sfixed(0,4,0);      -- ac_fixed
            <5,5>
22
23      function presize(
24          outtype:UNRESOLVED_sfixed;
25          arg:UNRESOLVED_sfixed)
26          return UNRESOLVED_sfixed is
27      begin
28          return resize(  arg,
29                          outtype'high,
30                          outtype'low,
31                          fixed_saturate,
32                          fixed_truncate
33                       );
34      end;
35
36      function quantize(arg : UNRESOLVED_sfixed) return UNRESOLVED_sfixed is
37          variable t : sfixed(4 downto 0);
38      begin
39          t := resize(arg           => arg,
40                      left_index    => 4,
41                      right_index   => 0,
42                      overflow_style => fixed_saturate,
43                      round_style   => fixed_truncate);
44          if t(0) = '0' then
45              t := resize(arg           => t + to_sfixed(1,4,0),
46                          left_index    => 4,
47                          right_index   => 0,
48                          overflow_style => fixed_saturate,
49                          round_style   => fixed_truncate);
50          end if;
51          return t;
52      end function quantize;
53
54  begin
55
56      process(all)
57          variable uin : sfixed(4 downto -11);   -- ac_fixed<16,5>
58          variable y1  : sfixed(4 downto   0);   -- ac_fixed<5,5>
59          variable e1  : sfixed(0 downto -11);   -- ac_fixed<12,1>
60          variable w1  : sfixed(4 downto  -7);   -- ac_fixed<12,5>
61          variable y2  : sfixed(4 downto   0);   -- ac_fixed<5,5>
62          variable e2  : sfixed(0 downto -11);   -- ac_fixed<12,1>
63          variable w2  : sfixed(4 downto -11);   -- ac_fixed<16,5>
64          variable v2  : sfixed(4 downto   0);   -- ac_fixed<5,5>
65      begin
```

```
66          if reset = '1' then
67              w1d <= to_sfixed(0,w1d);
68              w2d <= to_sfixed(0,w2d);
69              y1d <= to_sfixed(0,y1d);
70              v1  <= to_sfixed(0,v1);
71              y2d <= to_sfixed(0,y2d);
72              v2d <= to_sfixed(0,v2d);
73
74          elsif rising_edge(clk) then
75              uin := data_in;
76
77              -- ---------------- stepen 1 ----------------
78              y1 := quantize(w1d);
79              e1 := presize(e1, w1d - y1);
80              w1 := presize(w1, uin + e1);
81
82              -- ---------------- stepen 2 ----------------
83              y2 := quantize(w2d);
84              e2 := presize(e2, w2d - y2);
85              w2 := presize(w2, e1 + e2);
86
87              -- ------------------- H2 -------------------
88              v2 := presize(v2, y2 - y2d);
89
90              -- ---------------- registri ----------------
91              v1  <= y1d;
92              y1d <= y1;
93              w1d <= w1;
94              w2d <= w2;
95              y2d <= y2;
96              v2d <= v2;
97          end if;
98      end process;
99
100     vsum     <= presize(vsum, v1 + v2d);
101     data_out <= vsum;
102
103 end rtl;
```

Isečak koda B.4: Testbench simulacija modulatora

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use IEEE.Numeric_Std.all;
4  use IEEE.fixed_float_types.all;
5  use IEEE.fixed_pkg.all;
6  use STD.textio.all;
7
8  entity mash_tb is
9  end;
10
11 architecture bench of mash_tb is
12
13     component mash
14         Port(
15             data_in:in sfixed(4 downto -11);
16             reset,clk:in STD_LOGIC;
17             data_out:out sfixed(4 downto 0)
```

```
18              );
19      end component;
20
21      file infile  : text open read_mode is "Podaci";
22      file outfile : text open write_mode is "Obradjeni";
23      signal data_in:sfixed(4 downto -11);
24      signal reset,clk:std_logic;
25      signal data_out:sfixed(4 downto 0);
26      constant clock_period:time:=10 ns;
27      signal stop_the_clock:boolean:=false;
28      signal done  :boolean:=false;
29
30  begin
31      uut:MASH port map(
32                           data_in=>data_in,
33                           reset=>reset,
34                           clk=>clk,
35                           data_out=>data_out
36                      );
37
38      stimulus: process
39      begin
40          reset <= '1';
41          wait for clock_period;
42          reset <= '0';
43          wait for clock_period;
44          wait;
45      end process;
46
47      stim_proc : process
48          variable L : line;
49          variable d : real;
50      begin
51          data_in <= to_sfixed(0, data_in);
52          wait until reset = '0';
53          while not endfile(infile) loop
54              readline(infile, L);
55              read(L, d);
56              data_in <= to_sfixed(d, data_in, fixed_saturate, fixed_truncate
                    );
57              wait until rising_edge(clk);
58          end loop;
59          done <= true;
60          wait;
61      end process;
62
63      write_proc : process
64          variable L : line;
65      begin
66          wait until reset = '0';
67          loop
68              wait until rising_edge(clk);
69              wait for 1 ns;
70              write(L, to_integer(data_out));
71              writeline(outfile, L);
72              exit when done;
73          end loop;
74          stop_the_clock <= true;
```

```
75          file_close(infile);
76          file_close(outfile);
77          wait;
78      end process;
79
80      clocking : process
81          begin
82              while not stop_the_clock loop
83                  clk<='0','1' after clock_period/2;
84                  wait for clock_period;
85              end loop;
86              wait;
87          end process;
88  end;
```